Q3. [26 pts] Short Answers

(a) [5 pts] Given that (x_1, x_2) are jointly normally distributed with $\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}$ and $\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{bmatrix}$ ($\sigma_{21} = \sigma_{12}$), give an expression for the mean of the conditional distribution $p(x_1|x_2 = a)$.

This can be solved by writing $p(x_1|x_2 = a) = \frac{p(x_1, x_2=a)}{p(x_2=a)}$. x_2 being a component of a multivariate Gaussian is a univariate Gaussian with $x_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$. Write out the Gaussian densities and simplify (complete squares) to see the following:

$$x_1|x_2 = a \sim \mathcal{N}(\bar{\mu}, \bar{\sigma}^2), \quad \bar{\mu} = \mu_1 + \frac{\sigma_{12}}{\sigma_{22}^2}(a - \mu_2)$$

(b) [4 pts] The logistic function is given by $\sigma(x) = \frac{1}{1+e^{-x}}$. Show that $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

$$\sigma'(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{(1+e^{-x})} \cdot \frac{e^{-x}}{(1+e^{-x})} = \left(\frac{1}{1+e^{-x}}\right)\left(1 - \frac{1}{1+e^{-x}}\right) = \sigma(x)(1 - \sigma(x))$$

(c) Let X have a uniform distribution

$$p(x; \theta) = \begin{cases} \frac{1}{\theta} & 0 \leq x \leq \theta \\ 0 & \text{otherwise} \end{cases}$$

Suppose that n samples $x_1, \ldots, x_n$ are drawn independently according to $p(x; \theta)$.

(i) [5 pts] The maximum likelihood estimate of θ is $x_{(n)} = \max(x_1, x_2, \ldots, x_n)$. Show that this estimate of θ is biased.

Biased estimator: $\hat{\theta}$ (the sample estimate) is a biased estimator of θ (the population distribution parameter) if $E[\hat{\theta}] \neq \theta$.

Here $\hat{\theta} = x_{(n)}$. And $E[x_{(n)}] = \frac{n}{n+1}\theta \neq \theta$. The steps for finding $E[x_{(n)}]$ are given in the solutions of Homework 2, problem 5(c).

(ii) [2 pts] Give an expression for an unbiased estimator of θ.

$$\hat{\theta}_{unbiased} = \frac{n+1}{n} x_{(n)}$$

$$E[\hat{\theta}_{unbiased}] = E[\frac{n+1}{n} x_{(n)}] = \frac{n+1}{n} E[x_{(n)}] = \frac{n+1}{n} \times \frac{n}{n+1}\theta = \theta$$

(d) [5 pts] Consider the problem of fitting the following function to a dataset of 100 points $\{(x_i, y_i)\}$, $i = 1 \dots 100$:

$$y = \alpha cos(x) + \beta sin(x) + \gamma$$

This problem can be solved using the least squares method with a solution of the form:

$$\begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = (X^T X)^{-1} X^T Y$$

What are X and Y?

$$X = \begin{bmatrix} cos(x_1) & sin(x_1) & 1 \\ cos(x_2) & sin(x_2) & 1 \\ \vdots & \vdots & \vdots \\ cos(x_{100}) & sin(x_{100}) & 1 \end{bmatrix} \qquad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{100} \end{bmatrix}$$

(e) [5 pts] Consider the problem of binary classification using the Naive Bayes classifier. You are given two dimensional features (X_1, X_2) and the categorical class conditional distributions in the tables below. The entries in the tables correspond to $P(X_1 = x_1|C_i)$ and $P(X_2 = x_2|C_i)$ respectively. The two classes are *equally likely*.

$X_1 =$ \ Class	C_1	C_2
-1	0.2	0.3
0	0.4	0.6
1	0.4	0.1

$X_2 =$ \ Class	C_1	C_2
-1	0.4	0.1
0	0.5	0.3
1	0.1	0.6

Given a data point $(-1, 1)$, calculate the following posterior probabilities:

$P(C_1|X_1 = -1, X_2 = 1) =$ Using Bayes' Rule and conditional independence assumption of Naive Bayes

$$\frac{P(X_1=-1,X_2=1|C_1)P(C_1)}{P(X_1=-1,X_2=1)} = \frac{P(X_1=-1|C_1)P(X_2=1|C_1)P(C_1)}{P(X_1=-1|C_1)P(X_2=1|C_1)P(C_1)+P(X_1=-1|C_2)P(X_2=1|C_2)P(C_2)} = 0.1$$

$P(C_2|X_1 = -1, X_2 = 1) = 1 - P(C_1|X_2 = -1, X_1 = 1) = 0.9$

Q4. [10 pts] PCA and least squares

Recall that PCA transforms (zero-mean) data into low-dimensional reconstructions that lie in the span of the top k eigenvectors of the sample covariance matrix. Let $\mathbf{U}_k$ denote the $d \times k$ matrix of the top k eigenvectors of the covariance matrix ($\mathbf{U}_k$ is a truncated version of $\mathbf{U}$, which is the matrix of eigenvectors of the covariance matrix).

There are two approaches to computing the low-dimensional reconstruction $\mathbf{w} \in \mathbb{R}^k$ of a data point $\mathbf{x} \in \mathbb{R}^d$:

1. Solve a least squares problem to minimize the reconstruction error
2. Project $\mathbf{x}$ onto the span of the columns of $\mathbf{U}_k$

In this problem, you will show that these approaches are equivalent.

(a) [5 pts] Formulate the least squares problem in terms of $\mathbf{U}_k, \mathbf{x}$, and the variable $\mathbf{w}$.

(Hint: This optimization problem should resemble linear regression.)

We want to find the weights such that when we weight the columns of U_k, we will minimize the residual error. Thus, the objective function is $\|U_k w - x_i\|^2$. Here is our least squares problem:

$$\min_w \|U_k w - x_i\|^2$$

(b) [5 pts] Show that the solution of the least squares problem is equal to $\mathbf{U}_k^\top \mathbf{x}$, which is the projection of $\mathbf{x}$ onto the span of the columns of $\mathbf{U}_k$.

Recall the normal equations (which is the most important equation in the class!). For some unconstrained least squares problem in the form

$$\min_x \|Ax - y\|^2$$

the solution of the minimizer is

$$x^* = (A^T A)^{-1} A^T y$$

only when A is full rank. This applies here since U_k is full rank by definition. Plugging in our least squares problem, we have

$$w^* = (U_k^T U_k)^{-1} U_k^T x_i$$

If you don't remember this you can easily derive this by taking the gradient of the objective function and setting it to 0. We note that $U_k^T U_k = I_k$, (I_k = the identity matrix in k dimensions) thus

$$w^* = (U_k^T U_k)^{-1} U_k^T x_i = (I_k)^{-1} U_k^T x_i = U_k^T x_i$$

since the inverse of I is simply I.

Q5. [10 pts] Mixture of linear regressions

In class, you learned that K-means partitions points into K clusters by minimizing an objective that encourages points to be close to their cluster centers. K-means minimizes this objective in a coordinate descent fashion, alternating between computing cluster assignments and cluster centers until convergence.

In this problem, you will devise an algorithm, in the spirit of K-means, that fits an entirely different kind of mixture model. You are given a dataset $\{\mathbf{x}_i, y_i\}_{i=1}^N$, where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. You know that this dataset is a mixture of realizations of K different linear regression models

$$y = \mathbf{w}_k^\top \mathbf{x} + \mathcal{N}(0, \sigma^2)$$

parameterized by K weight vectors $\mathbf{w}_1, \ldots, \mathbf{w}_K \in \mathbb{R}^d$.

Your algorithm will jointly determine the following:

- A partition $S_1, \ldots, S_K$ of the dataset such that $(\mathbf{x}_i, y_i) \in S_k$ iff $(\mathbf{x}_i, y_i)$ comes from model k
- The model weights $\mathbf{w}_1, \ldots, \mathbf{w}_K$

(a) [4 pts] Write an objective function $f(S_1, \ldots, S_K, \mathbf{w}_1, \ldots, \mathbf{w}_K)$ to be minimized to solve this problem. Use the penalty $\|\mathbf{w}_k^\top \mathbf{x} - y\|^2$ if the point $(\mathbf{x}, y)$ is assigned to model k. Your objective should be a sum of N terms, and each data point should show up in exactly one of these terms.

$$f(S_1, \ldots, S_K, w_1, \ldots, w_K) = \sum_{k=1}^{K} \sum_{(x,y) \in S_k} \|w_k^\top x - y\|^2$$

(b) [3 pts] What is coordinate descent update for f with $\mathbf{w}_1, \ldots, \mathbf{w}_K$ fixed? In other words, to which of the K models should a point $(\mathbf{x}, y)$ be assigned?

Assign the point (x, y) to S_k if $k = \arg\min_k \|w_k^\top x - y\|^2$.

(c) [3 pts] Write the coordinate descent update for f with $S_1, \ldots, S_K$ fixed.

For a set S of $(\mathbf{x}, y)$ values, you should use the notation $\mathbf{X}_S$ to denote the design matrix whose rows are the $\mathbf{x}$-values of the elements of S, and $\mathbf{y}_S$ to denote the column vector of y-values of the elements of S.

$w_k = (X_{S_k}^\top X_{S_k})^{-1} X_{S_k}^\top y_{S_k}$

Q6. [10 pts] Training set augmentation

In class, you learned that one way to encourage invariance of a model to certain transformations is to augment the training set with extra examples perturbed according to those transformations. In this problem, you will examine the behavior of a certain type of input perturbation for a probabilistic linear regression setting.

Consider the following general generative model for regression:

- $\mathbf{x} \sim p(\mathbf{x})$, where $p(\mathbf{x})$ is a distribution over input vectors $\mathbf{x} \in \mathbb{R}^d$
- $y|\mathbf{x} \sim p(y|\mathbf{x})$, where $p(y|\mathbf{x})$ is a distribution over output scalars $y \in \mathbb{R}$ given $\mathbf{x}$

Assume that the relationship between y and $\mathbf{x}$ is well-modeled by a linear function $y = \mathbf{w}^\top \mathbf{x}$, where $\mathbf{w} \in \mathbb{R}^d$, so that in the infinite dataset limit, the objective to be minimized for this regression problem is:

$$\mathcal{L}_0(\mathbf{w}) = \mathbb{E}\left[(\mathbf{w}^\top \mathbf{x} - y)^2\right]$$

Now suppose the inputs are perturbed by zero-mean Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \lambda \mathbf{I})$, which is independent of the training data. The new objective is

$$\mathcal{L}(\mathbf{w}) = \mathbb{E}\left[(\mathbf{w}^\top (\mathbf{x} + \boldsymbol{\epsilon}) - y)^2\right]$$

(a) [9 pts] Compute and simplify $\mathcal{L}(\mathbf{w})$. Show all your work in detail, and write your answer in terms of $\mathcal{L}_0$.

$$\begin{aligned}
\mathcal{L}(w) &= \mathbb{E}\left[(w^\top (x + \epsilon) - y)^2\right] \\
&= \mathbb{E}\left[((w^\top x - y) + w^\top \epsilon)^2\right] \\
&= \mathbb{E}\left[(w^\top x - y)^2 + 2(w^\top x - y) w^\top \epsilon + w^\top \epsilon \epsilon^\top w\right] \\
&= \mathbb{E}\left[(w^\top x - y)^2\right] + \mathbb{E}\left[2(w^\top x - y) w^\top\right] \mathbb{E}[\epsilon] + w^\top \mathbb{E}\left[\epsilon \epsilon^\top\right] w
\end{aligned}$$

Substituting $\mathbb{E}[\epsilon] = 0$ and $\mathbb{E}[\epsilon \epsilon^\top] = \lambda I$, this simplifies to

$$= \mathcal{L}_0(w) + \lambda \|w\|^2$$

(b) [1 pt] Is there a relationship between this particular type of input perturbation and some type of regularization? If so, what kind of regularizer is involved?

In this setting, regression assuming this type of input perturbation turns out to be equivalent to regression with an L_2 regularizer.

Q7. [12 pts] Kernel PCA

You are given d-dimensional real-valued data $\{\mathbf{x}_i\}_{i=1}^N$ and a feature mapping $\phi : \mathbb{R}^d \to \mathbb{R}^m$. In the following questions, you will investigate how to do PCA in feature space on the feature vectors $\{\phi(\mathbf{x}_i)\}_{i=1}^N$. Assume that the data is centered in feature space; that is, $\sum_{i=1}^N \phi(\mathbf{x}_i) = 0$.

In the following, $\mathbf{\Phi}$ is a design matrix whose i^{th} row is $\phi(\mathbf{x}_i)$.

(a) [1 pt] Recall that as a part of PCA, we must solve the eigenvalue problem $\mathbf{Sv} = \lambda\mathbf{v}$, where $\mathbf{S}$ proportional to the sample covariance matrix. For PCA in feature space, we have $\mathbf{S} = \sum_{i=1}^N \phi(\mathbf{x}_i)\phi(\mathbf{x}_i)^\top = \mathbf{\Phi}^\top\mathbf{\Phi}$. Why is this a problem if m is large?

Working in feature space directly is too expensive if m is large. The covariance matrix $\Phi^\top\Phi$ is $m \times m$, which is too large to compute and work with.

(b) [3 pts] Now, you are given a kernel function $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$. Define the kernel matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

Show that if $\lambda \neq 0$, then λ is an eigenvalue of $\mathbf{S}$ if and only if λ is also an eigenvalue of $\mathbf{K}$ (in other words, finding feature-space principal components can be done by finding eigenvectors of $\mathbf{K}$).

Notice that $K = \Phi\Phi^\top$. The nonzero eigenvalues of $K = \Phi\Phi^\top$ and $S = \Phi^\top\Phi$ are the same.

(c) [4 pts] Let $\mathbf{v}$ be an eigenvector of $\mathbf{S}$ with nonzero eigenvalue λ. Show that $\mathbf{v}$ can be written as $\mathbf{v} = \mathbf{\Phi}^\top\boldsymbol{\alpha}_\mathbf{v}$, where $\boldsymbol{\alpha}_\mathbf{v}$ is an eigenvector of $\mathbf{K}$ with eigenvalue λ.

First, write $Sv = \Phi^\top\Phi v = \lambda v$. Multiplying this equation by Φ gives $K\alpha = \lambda\alpha$, where $\alpha = \Phi v$. Then, since $\Phi^\top\alpha = \Phi^\top\Phi v = Sv = \lambda v$, we have $v = \Phi^\top\alpha/\lambda$. Choosing $\alpha_v = \alpha/\lambda$ gives the desired result.

(d) [4 pts] You are given a new data point $\mathbf{x} \in \mathbb{R}^d$. Find the scalar projection of its feature representation $\phi(\mathbf{x})$ onto $\mathbf{v}/\|\mathbf{v}\|$ (with $\mathbf{v}$ defined as above).

Write your answer in terms of $\boldsymbol{\alpha}_\mathbf{v}$ and λ. Use the kernel k, and do not explicitly use ϕ. You should use the notation $\mathbf{k}_\mathbf{x} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}) & \cdots & k(\mathbf{x}_n, \mathbf{x}) \end{bmatrix}^\top$.

First, let's calculate the squared norm of v:

$$\|v\|^2 = v^\top v = \alpha_v^\top \Phi\Phi^\top \alpha_v = \alpha_v^\top K \alpha_v = \lambda\|\alpha_v\|^2$$

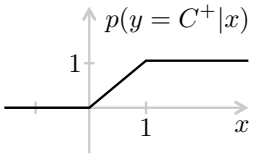
Then, the projection is:

$$\frac{v^\top\phi(x)}{\|v\|} = \frac{\alpha_v^\top\Phi\phi(x)}{\sqrt{\lambda}\|\alpha_v\|} = \frac{\alpha_v^\top k_x}{\sqrt{\lambda}\|\alpha_v\|}$$

Q3. [9 pts] Decision Theory

We are given a test that is designed to predict whether a patient y has cancer C^+ or not C^-. The test returns a value $x \in \mathcal{R}$ and we know the probability of the patient having cancer given the test results:

$$p(y = C^+|x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } 0 \leq x < 1 \\ 1, & \text{if } 1 \leq x \end{cases}$$



We also know that it is three times more costly to have a false negative than a false positive. Specifically, the loss matrix is:

Predicted:

Truth:		C^-	C^+
	C^-	0	10
	C^+	30	0

Suppose that we choose a fixed value x^*, and we predict C^+ if the test result is greater than x^* and C^- otherwise.

(a) [2 pts] What is the decision boundary (value of x) that minimizes the misclassification probability?

$x = 1/2$ minimizes the classification boundary.

We want to choose x such that the probability of C^+ given x is the same as the probability of C^- given x:

$$P(y = C^+|x) = P(y = C^-|x)$$
$$x = 1 - x$$
$$x = 1/2$$

(b) [3 pts] What is the decision boundary (value of x^*) that minimizes the risk?

$x^* = 1/4$ minimizes the risk.

Choose x^* such that the risk of choosing C^+ given x is the same as the risk of choosing of C^- given x:

$$R(\hat{y} = C^+|x^*) = R(\hat{y} = C^-|x^*)$$
$$\ell(C^+, C^+)P(y = C^-|x^*) + \ell(C^+, C^-)P(y = C^-|x^*) = \ell(C^-, C^+)P(y = C^+|x^*) + \ell(C^-, C^-)P(y = C^-|x^*)$$
$$\ell(C^+, C^-)P(y = C^-|x^*) = \ell(C^-, C^+)P(y = C^+|x^*)$$
$$10(1 - x^*) = 30x^*$$
$$x^* = 1/4$$

(c) [4 pts] If the test result is uniformly distributed in the interval $[-1, 1]$, what is the value of the minimum risk? Write your answer in terms of x^* (to avoid loss of points if your x^* is incorrect).

$$
\begin{aligned}
\mathrm{E}\ell(\hat{y}, y) &= \mathrm{EE}[\ell(\hat{y}, y)|x] \\
&= \int_{-\infty}^{\infty} \mathbb{1}[\hat{y} = C^-]\ell(C^-, C^+)P(y = C^+, x) + \mathbb{1}[\hat{y} = C^+]\ell(C^+, C^-)P(y = C^-, x)dx \\
&= \int_{-\infty}^{\infty} \mathbb{1}[\hat{y} = C^-]\ell(C^-, C^+)P(y = C^+|x)P(x) + \mathbb{1}[\hat{y} = C^+]\ell(C^+, C^-)P(y = C^-|x)P(x)dx \\
&= \int_{0}^{x^*} \frac{1}{2}\ell(C^-, C^+)P(y = C^+|x)dx + \int_{x^*}^{1} \frac{1}{2}\ell(C^+, C^-)P(y = C^-|x)dx \\
&= \frac{1}{2}\int_{0}^{x^*} 30x dx + \int_{x^*}^{1} 10(1 - x)dx \\
&= \frac{1}{2}\left(15x^2\Big]_0^{x^*} + 10x - 5x^2\Big]_{x^*}^{1}\right) \\
&= \frac{1}{2}\left(15x^{*2} + 5 - 10x^* + 5x^{*2}\right) \\
&= 10x^{*2} - 5x^* + 2.5 = \frac{15}{8}
\end{aligned}
$$

Q4. [8 pts] Parameter Estimation

Suppose you are given n observations, $X_1, ..., X_n$, independent and identically distributed with a Gamma(α, λ) distribution. The following information might be useful for one or more parts of the problem.

- If $X \sim \text{Gamma}(\alpha, \lambda)$, then $\mathbb{E}[X] = \frac{\alpha}{\lambda}$ and $\mathbb{E}[X^2] = \frac{\alpha(\alpha+1)}{\lambda^2}$
- The probability density function of $X \sim \text{Gamma}(\alpha, \lambda)$ is $f_X(x) = \frac{1}{\Gamma(\alpha)}\lambda^\alpha x^{\alpha-1}e^{-\lambda x}$ where the function Γ is only dependent on α and not λ.

The following notation might be useful for one or more parts of the problem: $\overline{X}_1 = \frac{1}{n}\sum_{i=1}^{n} X_i$ and $\overline{X}_2 = \frac{1}{n}\sum_{i=1}^{n} X_i^2$.

(a) [4 pts] Find the estimators for α and λ using the method of moments. Remember you are trying to write α and λ as functions of the data.

First, we take the sample moments from the data, $\overline{X}_1, \overline{X}_2$ and equate that to the theoretical moments, $\mathbb{E}[X], \mathbb{E}[X^2]$.

$$\overline{X}_1 = \frac{\alpha}{\lambda}$$
$$\overline{X}_2 = \frac{\alpha(\alpha+1)}{\lambda^2}$$

Solving for α and λ from this system of equations, we get

$$\hat{\alpha}_{\text{MM}} = \frac{\overline{X}_1^2}{\overline{X}_2 - \overline{X}_1^2} = \frac{\overline{X}_1^2}{\hat{\sigma}^2}$$
$$\hat{\lambda}_{\text{MM}} = \frac{\overline{X}_1}{\overline{X}_2 - \overline{X}_1^2} = \frac{\overline{X}_1}{\hat{\sigma}^2}$$

(b) [4 pts] Suppose, we are given a known, fixed value for α. Compute the maximum likelihood estimator for λ.

We first write the likelihood function.

$$L(\lambda | X_1, ..., X_n) = \prod_{i=1}^{n} \frac{1}{\Gamma(\alpha)}\lambda^\alpha X_i^{\alpha-1}e^{-\lambda X_i}$$

The log-likelihood function is given as follows.

$$l(\lambda | X_1, ..., X_n) = -n\log(\Gamma(\alpha)) + n\alpha\log\lambda + \sum_{i=1}^{n}(\alpha-1)\log X_i - \lambda X_i$$

Next, we take the gradient with respect to λ and set it equal to 0.

$$\nabla_\lambda l(\lambda | X_1, ..., X_n) = \frac{n\alpha}{\lambda} - \sum_{i=1}^{n} X_i = 0$$

Solving for λ, we get,

$$\hat{\lambda}_{\text{MLE}} = \frac{n\alpha}{\sum_{i=1}^{n} X_i} = \frac{\alpha}{\frac{1}{n}\sum_{i=1}^{n} X_i} = \frac{\alpha}{\overline{X}_1}$$

Q5. [14 pts] Locally Weighted Logistic Regression

In this problem, we consider solving the problem of locally weighted logistic regression. Given data $\{(x_i, y_i) \in \mathbb{R}^d \times \{0, 1\}\}_{i=1}^n$ and a query point x, we choose a parameter vector θ to minimize the loss (which is simply the negative log likelihood, weighted appropriately):

$$l(\theta; x) = -\sum_{i=1}^{n} w_i(x) \left[y_i \log(\mu(x_i)) + (1 - y_i) \log(1 - \mu(x_i)) \right]$$

where

$$\mu(x_i) = \frac{1}{1 + e^{-\theta \cdot x_i}}, \qquad w_i(x) = \exp\left(-\frac{\|x - x_i\|^2}{2\tau} \right)$$

where τ is a hyperparameter that must be tuned. Note that whenever we receive a new query point x, we must solve the entire problem again with these new weights $w_i(x)$.

Hint: the derivative of the logistic regression log likelihood with respect to θ is: $\sum_{i=1}^{n}(y_i - \mu(x_i))x_i$

(a) [4 pts] Given a data point x, derive the gradient of $l(\theta; x)$ with respect to θ.

The derivation is extremely similar to the derivation of logistic regression as in the slides. The answer is

$$-X^\intercal z = -\sum_{i=1}^{n} w_i(x)(y_i - \mu(x_i))x_i$$

where X is the design matrix (i.e. every row in X is a data point), and $z_i = w_i(x)(y_i - \mu(x_i))$.

(b) [4 pts] Given a data point x, derive the Hessian of $l(\theta; x)$ with respect to θ.

The derivation is extremely similar to the derivation of logistic regression as in the slides. The answer is

$$X^T D X = \sum_{i=1}^{n} w_i(x)\mu(x_i)(1 - \mu(x_i))x_i x_i^\intercal$$

(c) [2 pts] Given a data point x, write the update formula for gradient descent. Use the symbol η for an arbitrary step size.

$$\theta^{(t+1)} = \theta^{(t)} + \eta X^\intercal z$$

(d) [2 pts] Given a data point x, write the update formula for Newton's method.

$$\theta^{(t+1)} = \theta^{(t)} + [X^T D X]^{-1} X^\intercal z$$

(e) [2 pts] Locally Weighted Logistic Regression is a

○ Parametric method ● Nonparametric method

Q6. [7 pts] Decision Trees

Answer the following questions related to decision trees.

(a) [3 pts] In Homework 5, you first implemented a decision tree and then implemented a decision forest, which uses an ensemble method called bagging.

For neural network classification, it is typical to train k networks and average the results. Why not run your decision tree (using all of the data) k times and then average the results?

Decision trees return the same result each time you run them.

(b) [2 pts] True or false: Selecting the decision tree split (at each node as you move down the tree) that *minimizes classification error* will guarantee an optimal decision tree.

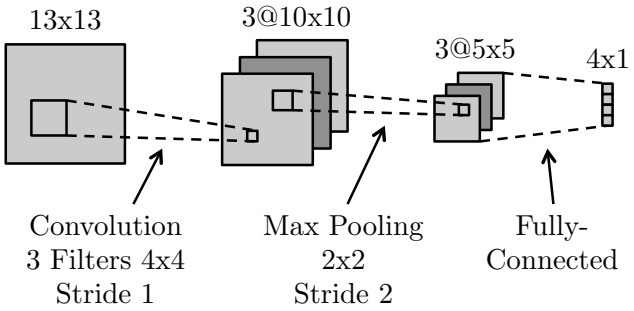
○ True ● False

(c) [2 pts] True or false: Selecting the decision tree split (at each node as you move down the tree) that *maximizes information gain* will guarantee an optimal decision tree.

○ True ● False

Q7. [11 pts] Convolutional Neural Nets

Below is a diagram of a small convolutional neural network that converts a 13x13 image into 4 output values. The network has the following layers/operations from input to output: convolution with 3 filters, max pooling, ReLu, and finally a fully-connected layer. For this network we will not be using any bias/offset parameters (b). Please answer the following questions about this network.



(a) [2 pts] How many weights in the convolutional layer do we need to learn?

48 weights. Three filters with 4x4=16 weights each.

(b) [2 pts] How many ReLu operations are performed on the forward pass?

75 ReLu operations. ReLu is performed after the pooling step. ReLu is performed on each pixel of the three 5x5 feature images.

(c) [2 pts] How many weights do we need to learn for the entire network?

348 weights. 48 for the convolutional layer. Fully-connected has 3x5x5=75 pixels each connected to four outputs, which is 300 weights. Pooling layer does not have any weights.

(d) [2 pts] True or false: A fully-connected neural network with the same size layers as the above network (13x13 $\rightarrow$ 3x10x10 $\rightarrow$ 3x5x5 $\rightarrow$ 4x1) can represent any classifier that the above convolutional network can represent.

● True ○ False

(e) [3 pts] What is the disadvantage of a fully-connected neural network compared to a convolutional neural network with the same size layers?

Too many weights to effectively learn.

Q8. [9 pts] Streaming k-means

The standard k-means algorithm loads all data points altogether into the memory. In practice, data usually comes in a stream, such that they are sequentially processed and dropped (not stored in memory). The advantage of streaming algorithms is that their memory requirement is independent of the stream length. Thus, streaming algorithms are very useful in processing data that cannot fit into the memory.

In this problem, we will explore how to extend the k-means algorithm to process streaming data. Suppose that there are k clusters. The cluster centers are randomly initialized. Once the processor receives a data point $x \in \mathbb{R}^d$, it does the following:

1. Find the cluster whose center is the closest to x (in Euclidean distance), then add x to the cluster
2. Adjust the cluster center so that it equals the mean of all cluster members.

The algorithm outputs the k cluster centres after processing all data points in the stream.

According to the above algorithm specification, complete the streaming algorithm for k-means. Note that the algorithm's memory requirement should be independent of the stream length.

(a) [3 pts] List the variables that are stored in the memory and their initial values. Which variables should be the output of the algorithm?

Two sets of variables need to be stored:

- c_i: the center of the i-th cluster ($i = 1, \ldots, k$). It is a d-dimensional vector and should be randomly initialized in $\mathbb{R}^d$.
- n_i: the number of data points that belong to the i-th cluster. It should be initialized by 0.

The algorithm's output should be c_i ($i = 1, \ldots, k$).

(b) [3 pts] When the processor receives a data point x, state the updates that are made on the variables.

The algorithm executes the following three steps:

1. Find the index i which minimizes $\|x - c_i\|_2$ for $i = 1, \ldots, k$.
2. Update $c_i \leftarrow (n_i c_i + x)/(n_i + 1)$.
3. Update $n_i \leftarrow n_i + 1$.

(c) [3 pts] In each iteration, suppose the processor receives a data point x along with its weight $w > 0$. We want the cluster center to be the weighted average of all cluster members. How do you modify the updates in question (b) to process weighted data?

With weighted data, the algorithm executes the following three steps:

1. Find the index i which minimizes $\|x - c_i\|_2$ for $i = 1, \ldots, k$.
2. Update $c_i \leftarrow (n_i c_i + wx)/(n_i + w)$.
3. Update $n_i \leftarrow n_i + w$.

Q3. [15 pts] Nearest Neighbors and Bayes risk

In this problem, we want to investigate whether given enough training examples, the Bayes decision rule gives more accurate results that nearest neighbors.

A life insurance company needs to estimate whether a client is at risk of dying in the year to come, based on his age and blood pressure. We call $\mathbf{x} = [A, B]$ (A=Age, B=Blood pressure) the two dimensional input vector and y the outcome ($y = 1$ if the client dies and $y = -1$ otherwise). The insurance company has a lot of data, enough to estimate accurately with Parzen windows the posterior probability $P(y = 1|\mathbf{x})$. This is represented in a diagram in Figure 1.

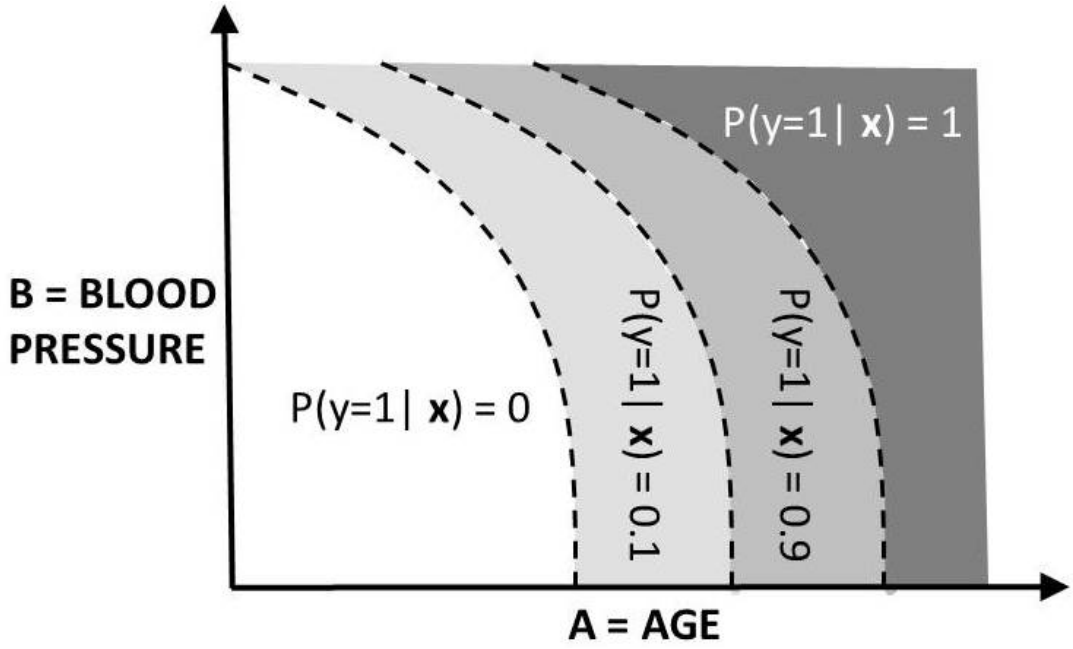


Figure 1: Draw your answer to the Nearest Neighbor and Bayes risk problem. Note that the distribution $P(x)$ is assumed to be uniform across the area shown in the figure.

Note: No worries, nobody died, this is a fictitious example. For simplicity we have 4 regions in which $P(y = 1|\mathbf{x})$ is constant, and the distribution $P(x)$ is assumed to be uniform across the area shown in the figure.

Let us name the different regions:

$$R1 : P(y = 1|\mathbf{x}) = 0$$
$$R2 : P(y = 1|\mathbf{x}) = 0.1$$
$$R3 : P(y = 1|\mathbf{x}) = 0.9$$
$$R4 : P(y = 1|\mathbf{x}) = 1$$

(1) [2 pts] Draw on the figure the Bayes optimum decision boundary with a thick line.

(2) [2 pts] What is the Bayes risk in each of the four regions (the Bayes risk is the probability of error of the optimum Bayes classifier). R1: EBayes = 0
R2: EBayes = 0.1
R3: EBayes = 0.1
R4: EBayes = 0

(3) [4 pts] Assume we have lots and lots of samples due to which we can assume that the nearest neighbor of any sample lies in the same region as that sample. Now consider any sample, say, $\mathbf{x}$ which falls in region R_i. For $i \in \{1, 2, 3, 4\}$, find the probability that $\mathbf{x}$ and its nearest neighbor belong to different classes (that is, have

different labels):

1. If they both fall in R1: 0
2. If they both fall in R2: x 0.9 (+1) x 0.1 (+1) + x 0.1 (+1) x 0.9 (+1) $\Rightarrow$ 2 * 0.1 * 0.9 = 0.18
3. If they both fall in R3: same as R2: 2 * 0.1 * 0.9
4. If they both fall in R4: 0

(4) [2 pts] What is the nearest neighbor error rate ENN in each region:

R1: ENN = 0
R2: ENN = 0.18
R3: ENN = 0.18
R4: ENN = 0

(5) [5 pts] Now let us generalize the previous results to the case where the posterior probabilities are:

$$R1 : P(y = 1|\mathbf{x}) = 0$$
$$R2 : P(y = 1|\mathbf{x}) = p$$
$$R3 : P(y = 1|\mathbf{x}) = (1 - p)$$
$$R4 : P(y = 1|\mathbf{x}) = 1$$

where p is a number between 0 and 0.5.
After recalculating the results of the previous questions, give an upper bound and a lower bound of ENN in terms of EBayes.

$$R1 : EBayes = 0 ENN = 0$$
$$R2 : EBayes = p ENN = 2p(1 - p)$$
$$R3 : EBayes = p ENN = 2p(1 - p)$$
$$R4 : EBayes = 0 ENN = 0$$

EBayes <= ENN <= 2 EBayes(1-EBayes)

Q5. [22 pts] Decision Trees and Random Forests

Consider constructing a decision tree on data with d features and n training points where each feature is real-valued and each label takes one of m possible values. The splits are two-way, and are chosen to maximize the information gain. We only consider splits that form a linear boundary parallel to one of the axes. In parts (a), (b) and (c) we will consider a standalone decision tree and not a random forest, so no randomization.

(1) [4 pts] Prove or give a counter-example: For every value of $m > 3$, there exists some probability distribution on m objects such that its entropy is less than -1. False. The entropy is always non-negative since $-p \log p$ is non-negative when $p \in [0, 1]$.

(2) [4 pts] Prove or give a counter-example: In any path from the root split to a leaf, the same feature will never be split on twice.

False. Example: one dimensional feature space with training poins of two classes x and o arranged as xxxooooxxx.

(3) [4 pts] Prove or give a counter-example: The information gain at the root is at least as much as the information gain at any other node.

False. Example: the XOR function.

(4) [4 pts] One may be concerned that the randomness introduced in random forests may cause trouble, for instance, some features or samples may not be considered at all. We will investigate this phenomenon in the next two parts.

Consider n training points in a feature space of d dimensions. Consider building a random forest with t binary trees, each having exactly h internal nodes. Let f be the number of features randomly selected at each node. In order to simplify our calculations, we will let $f = 1$. For this setting, compute the probability that a certain feature (say, the first feature) is never considered for splitting.

The probability that it is not considered for splitting in a particluar node of a particular tree is $1 - \frac{1}{d}$. The subsampling of $f = 1$ features at each node is independent of all others. There are a total of th nodes and hence the final answer is $(1 - \frac{1}{d})^{th}$.

(5) [3 pts] Now let us investigate the concern regarding the random selection of the samples. Suppose each tree employs n bootstrapped training samples. Compute the probability that a particular sample (say, the first sample) is never considered in any of the trees.

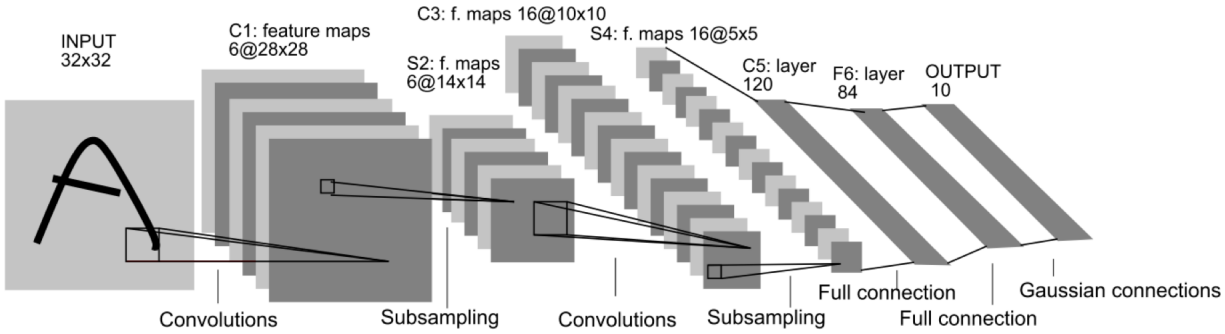
The probability that it is not considered in one of the trees is $(1 - \frac{1}{n})^n$. Since the choice for every tree is independent, the probability that it is not considered in any of the trees is $(1 - \frac{1}{n})^{nt}$.

(6) [3 pts] Compute the values of the probabilities you obtained in the previous two parts for the case when there are $n = 2$ training points, $d = 2$ dimensions, $t = 10$ trees of depth $h = 4$ (you may leave your answer in a fraction and exponentiated form, e.g., as $\left(\frac{51}{100}\right)^2$. What conclusions can you draw from your answer with regard to the concern mentioned in the beginning of the problem?

$\frac{1}{2^{150}}$ and $\frac{1}{2^{20}}$. It is quite unlikely that a feature or a sample will be missed.

Q7. [14 pts] Neural Networks

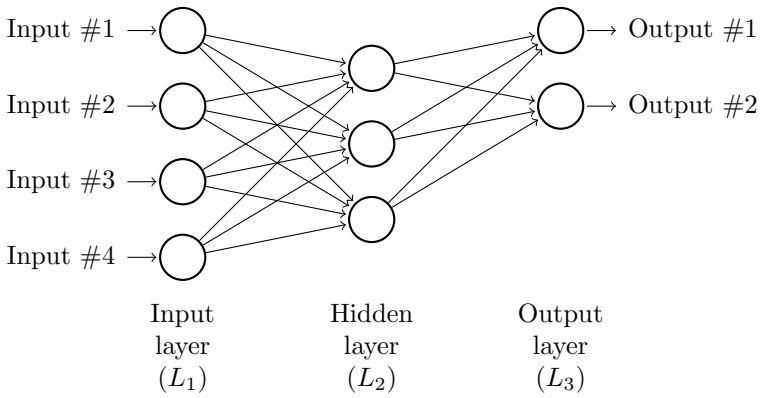
(1) [6 pts] Here is the historical *LeNet* Convolutional Neural Network architecture of Yann LeCun et al. for digit classification that we've discussed in class. Here, the INPUT layer takes in a 32x32 image, and the OUTPUT layer produces 10 outputs. The notation 6@28x28 means 6 matrices of size 28x28.



If the parameters of a given layer are the weights that connect to its inputs,

- Given that the input size is 32x32, and the Layer 1 size is 28x28, what's the size of the convolutional filter in the first layer (i.e. how many inputs is each neuron connected to)? 5×5
- How many independent parameters (weight and bias) are in layer C1? $5 \times 5 \times 6 \times 1 + 6 = 156$
- How many independent parameters (weight and bias) are in layer C3? $5 \times 5 \times 16 \times 6 + 16 = 2416$
- How many independent parameters (weight and bias) are in layer F6? $120 \times 84 + 84 = 10164$

(2) [8 pts] Consider a three layer fully-connected network with n_1, n_2, n_3 neurons in three layers respectively. Inputs are fed into the first layer. The loss is mean squared error E, and the non-linearity is a sigmoid function. Let the label vector be t of size n_3. Let each layer output vector be y_i and input vector be z_i, both of size n_i. Let the weight between layer i and layer $i+1$ be W_{ii+1}. The j-th element in y_i is defined by y_i^j, same for z_i^j. The weight connecting k-th and l-th neuron in $i, i+1$ layers is defined by W_{ii+1}^{kl} (You don't need to consider bias in this problem).



Here is a summary of our notation:

- σ denotes the activation function for L_2 and L_3, $\sigma(x) = \frac{1}{1+e^{-x}}$. There is no activation applied to the input layer.
- $z_i^{(j)} = \sum_{k=1}^{P} W_{i-1i}^{kj} x_{i-1}^{(k)}$
- $y_i^{(j)} = \sigma\left(\sum_{k=1}^{P} W_{i-1i}^{kj} x_{i-1}^{(k)}\right)$

Now solve the following problems.

- Find $\frac{\partial E}{\partial z_3^j}$ in terms of y_3^j.
  $-2y_3^j(1-y_3^j)(t^j - y_3^j)$
- Find $\frac{\partial E}{\partial y_2^k}$ in terms of elements in W_{23} and $\frac{\partial E}{\partial z_3^j}$.
  $\sum_{j=1}^{n_3} W_{23}^{kj} \frac{\partial E}{\partial z_3^j}$
- Find $\frac{\partial E}{\partial W_{23}^{kj}}$ in terms of y_2^k, y_3^j and t^j.
  $y_2^k \frac{\partial E}{\partial z_3^j} = -2y_3^j(1-y_3^j)(t^j - y_3^j)y_2^k$
- If the input to a neuron in max-pooling layer is x and the output is $y = max(x)$, derive $\frac{\partial y}{\partial x_i}$.
  $\frac{\partial y}{\partial x_i} = 1$ if and only if $x_i = max(x)$, otherwise $\frac{\partial y}{\partial x_i} = 0$.

Q3. [10 pts] Gradient Descent for k-means Clustering

Recall the loss function for k-means clustering with k clusters, sample points $x_1, ..., x_n$, and centers $\mu_1, ..., \mu_k$:

$$L = \sum_{j=1}^{k} \sum_{x_i \in S_j} \|x_i - \mu_j\|^2,$$

where S_j refers to the set of data points that are closer to μ_j than to any other cluster mean.

(1) [4 pts] Instead of updating μ_j by computing the mean, let's minimize L with **batch** gradient descent while holding the sets S_j fixed. Derive the update formula for μ_1 with learning rate (step size) ϵ.

$$\begin{aligned} \frac{\partial L}{\partial \mu_1} &= \frac{\partial}{\partial \mu_1} \sum_{x_i \in S_1} (x_i - \mu_1)^\top (x_i - \mu_1) \\ &= \sum_{x_i \in S_1} 2(\mu_1 - x_i). \end{aligned}$$

Therefore the update formula is

$$\mu_1 \leftarrow \mu_1 + \epsilon \sum_{x_i \in S_1} (x_i - \mu_1).$$

(Note: writing 2ϵ instead of ϵ is fine.)

(2) [2 pts] Derive the update formula for μ_1 with **stochastic** gradient descent on a single sample point x_i. Use learning rate ϵ.

$\mu_1 \leftarrow \mu_1 + \epsilon(x_i - \mu_1)$ if $x_i \in S_1$, otherwise no change.

(3) [4 pts] In this part, we will connect the batch gradient descent update equation with the standard k-means algorithm. Recall that in the update step of the standard algorithm, we assign each cluster center to be the mean (centroid) of the data points closest to that center. It turns out that a particular choice of the learning rate ϵ (which may be different for each cluster) makes the two algorithms (batch gradient descent and the standard k-means algorithm) have identical update steps. Let's focus on the update for the first cluster, with center μ_1. Calculate the value of ϵ so that both algorithms perform the same update for μ_1. (If you do it right, the answer should be very simple.)

In the standard algorithm, we assign $\mu_1 \leftarrow \sum_{x_i \in S_1} \frac{1}{|S_1|} x_i$.

Comparing to the answer in (1), we set $\sum_{x_i \in S_1} \frac{1}{|S_1|} x_i = \mu_1 + \epsilon \sum_{x_i \in S_1} (x_i - \mu_1)$ and solve for ϵ.

$$\begin{aligned} \sum_{x_i \in S_1} \frac{1}{|S_1|} x_i - \sum_{x_i \in S_1} \frac{1}{|S_1|} \mu_1 &= \epsilon \sum_{x_i \in S_1} (x_i - \mu_1) \\ \sum_{x_i \in S_1} \frac{1}{|S_1|} (x_i - \mu_1) &= \epsilon \sum_{x_i \in S_1} (x_i - \mu_1). \end{aligned}$$

Thus $\epsilon = \frac{1}{|S_1|}$.

(Note: answers that differ by a constant factor are fine if consistent with answer for (1).)

Q4. [10 pts] Kernels

(1) [2 pts] What is the primary motivation for using the kernel trick in machine learning algorithms?

If we want to map sample points to a very high-dimensional feature space, the kernel trick can save us from having to compute those features explicitly, thereby saving a lot of time.

(Alternative solution: the kernel trick enables the use of infinite-dimensional feature spaces.)

(2) [4 pts] Prove that for every design matrix $X \in \mathbb{R}^{n \times d}$, the corresponding kernel matrix is positive semidefinite.

For every vector $\mathbf{z} \in \mathbb{R}^n$,

$$\mathbf{z}^\top K \mathbf{z} = \mathbf{z}^\top X X^\top \mathbf{z} = |X^\top \mathbf{z}|^2,$$

which is clearly nonnegative.

(3) [2 pts] Suppose that a regression algorithm contains the following line of code.

$$\mathbf{w} \leftarrow \mathbf{w} + X^\top M X X^\top \mathbf{u}$$

Here, $X \in \mathbb{R}^{n \times d}$ is the design matrix, $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $M \in \mathbb{R}^{n \times n}$ is a matrix unrelated to X, and $\mathbf{u} \in \mathbb{R}^n$ is a vector unrelated to X. We want to derive a dual version of the algorithm in which we express the weights $\mathbf{w}$ as a linear combination of samples X_i (rows of X) and a dual weight vector $\mathbf{a}$ contains the coefficients of that linear combination. Rewrite the line of code in its dual form so that it updates $\mathbf{a}$ correctly (and so that $\mathbf{w}$ does not appear).

$$\mathbf{a} \leftarrow \mathbf{a} + M X X^\top \mathbf{u}$$

(4) [2 pts] Can this line of code for updating $\mathbf{a}$ be kernelized? If so, show how. If not, explain why.

Yes:

$$\mathbf{a} \leftarrow \mathbf{a} + M K \mathbf{u}$$

Q5. [12 pts] Let's PCA

You are given a design matrix $X = \begin{bmatrix} 6 & -4 \\ -3 & 5 \\ -2 & 6 \\ 7 & -3 \end{bmatrix}$. Let's use PCA to reduce the dimension from 2 to 1.

(1) [6 pts] Compute the covariance matrix for the sample points. (Warning: Observe that X is not centered.) Then compute the **unit** eigenvectors, and the corresponding eigenvalues, of the covariance matrix. *Hint:* If you graph the points, you can probably guess the eigenvectors (then verify that they really are eigenvectors).

The covariance matrix is $X^\top X = \begin{bmatrix} 82 & -80 \\ -80 & 82 \end{bmatrix}$.

Its unit eigenvectors are $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$ with eigenvalue 2 and $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{bmatrix}$ with eigenvalue 162. (Note: either eigenvector can be replaced with its negation.)

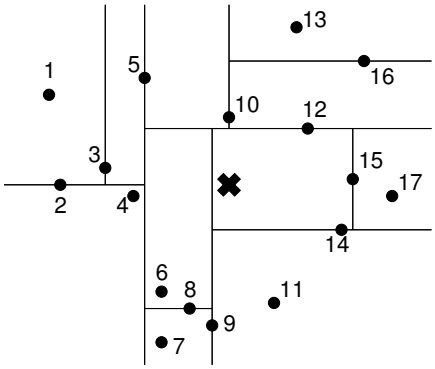
(2) [3 pts] Suppose we use PCA to project the sample points onto a one-dimensional space. What one-dimensional subspace are we projecting onto? For each of the four sample points in X (not the centered version of X!), write the coordinate (in principal coordinate space, not in $\mathbb{R}^2$) that the point is projected to.

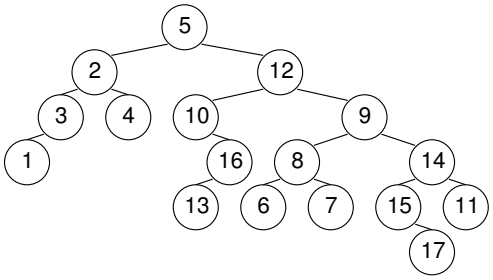
We are projecting onto the subspace spanned by $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{bmatrix}$. (Equivalently, onto the space spanned by $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Equivalently, onto the line $x + y = 0$.) The projections are $(6, -4) \to \frac{10}{\sqrt{2}}$, $(-3, 5) \to -\frac{8}{\sqrt{2}}$, $(-2, 6) \to -\frac{8}{\sqrt{2}}$, $(7, -3) \to \frac{10}{\sqrt{2}}$.

(3) [3 pts] Given a design matrix X that is taller than it is wide, prove that every right singular vector of X with singular value σ is an eigenvector of the covariance matrix with eigenvalue σ^2.

If v is a right singular vector of X, then there is a singular value decomposition $X = UDV^\top$ such that v is a column of V. Here each of U and V has orthonormal columns, V is square, and D is square and diagonal. The covariance matrix is $X^\top X = VDU^\top UDV^\top = VD^2V^\top$. This is an eigendecomposition of $X^\top X$, so each singular vector in V with singular value σ is an eigenvector of $X^\top X$ with eigenvalue σ^2.

Q6. [10 pts] Trees





(1) [5 pts] Above, we have two depictions of the same k-d tree, which we have built to solve nearest neighbor queries. Each node of the tree at right represents a rectangular box at left, and also stores one of the sample points that lie inside that box. (The root node represents the whole plane $\mathbb{R}^2$.) If a treenode stores sample point i, then the line passing through point i (in the diagram at left) determines which boxes the child treenodes represent.

Simulate running an exact 1-nearest neighbor query, where the bold X is the query point. Recall that the query algorithm visits the treenodes in a smart order, and keeps track of the nearest point it has seen so far.

- Write down the numbers of all the sample points that serve as the "nearest point seen so far" sometime while the query algorithm is running, in the order they are encountered.
- Circle all the subtrees in the k-d tree at upper right that are never visited during this query. (This is why k-d tree search is usually faster than exhaustive search.)

Nearest point seen so far: first 5, then 12, then 10.

The unvisited subtrees are rooted at 2, 13, 7, and 17.

(2) [5 pts] We are building a decision tree for a 2-class classification problem. We have n training points, each having d real-valued features. At each node of the tree, we try every possible univariate split (i.e. for each feature, we try every possible splitting value for that feature) and choose the split that maximizes the information gain.

Explain why it is possible to build the tree in $O(ndh)$ time, where h is the depth of the tree's deepest node. Your explanation should include an analysis of the time to choose one node's split. Assume that we can radix sort real numbers in linear time.

Consider choosing the split at a node whose box contains n' sample points. For each of the d features, we can sort the sample points in $O(n'd)$ time. Then we can compute the entropy for the first split (separating the first sample in the sorted list from the others) in $O(n')$ time, then we can walk through the list and update the entropy for each successive split in $O(1)$ time, summing to a total of $O(n')$ time for each of the d features. So it takes $O(n'd)$ time overall to choose a split.

Each sample point participates in at most h treenodes, so each sample point contributes at most dh to the running time, for a total running time of at most $O(ndh)$.

Q7. [10 pts] Self-Driving Cars and Backpropagation

You want to train a neural network to drive a car. Your training data consists of grayscale 64×64 pixel images. The training labels include the human driver's steering wheel angle in degrees and the human driver's speed in miles per hour. Your neural network consists of an input layer with $64 \times 64 = 4{,}096$ units, a hidden layer with 2,048 units, and an output layer with 2 units (one for steering angle, one for speed). You use the ReLU activation function for the hidden units and no activation function for the outputs (or inputs).

(1) [2 pts] Calculate the number of parameters (weights) in this network. You can leave your answer as an expression. Be sure to account for the bias terms.

$$4097 \times 2048 + 2049 \times 2$$

(2) [3 pts] You train your network with the cost function $J = \frac{1}{2}|\mathbf{y} - \mathbf{z}|^2$. Use the following notation.

- $\mathbf{x}$ is a training image (input) vector with a 1 component appended to the end, $\mathbf{y}$ is a training label (input) vector, and $\mathbf{z}$ is the output vector. All vectors are column vectors.
- $r(\gamma) = \max\{0, \gamma\}$ is the ReLU activation function, $r'(\gamma)$ is its derivative (1 if $\gamma > 0$, 0 otherwise), and $r(\mathbf{v})$ is $r(\cdot)$ applied component-wise to a vector.
- $\mathbf{g}$ is the vector of hidden unit values before the ReLU activation functions are applied, and $\mathbf{h} = r(\mathbf{g})$ is the vector of hidden unit values after they are applied (but we append a 1 component to the end of $\mathbf{h}$).
- V is the weight matrix mapping the input layer to the hidden layer; $\mathbf{g} = V\mathbf{x}$.
- W is the weight matrix mapping the hidden layer to the output layer; $\mathbf{z} = W\mathbf{h}$.

Derive $\partial J / \partial W_{ij}$.

$$\begin{aligned} \frac{\partial J}{\partial W_{ij}} &= (\mathbf{z} - \mathbf{y})^\top \frac{\partial \mathbf{z}}{\partial W_{ij}} \\ &= (\mathbf{z}_i - \mathbf{y}_i)\mathbf{h}_j \end{aligned}$$

(3) [1 pt] Write $\partial J / \partial W$ as an outer product of two vectors. $\partial J / \partial W$ is a matrix with the same dimensions as W; it's just like a gradient, except that W and $\partial J / \partial W$ are matrices rather than vectors.

$$\frac{\partial J}{\partial W} = (\mathbf{z} - \mathbf{y})\mathbf{h}^\top$$

(4) [4 pts] Derive $\partial J / \partial V_{ij}$.

$$\begin{aligned} \frac{\partial J}{\partial V_{ij}} &= (\mathbf{z} - \mathbf{y})^\top \frac{\partial \mathbf{z}}{\partial V_{ij}} \\ &= (\mathbf{z} - \mathbf{y})^\top W \frac{\partial \mathbf{h}}{\partial V_{ij}} \\ &= (\mathbf{z} - \mathbf{y})^\top W \left[0, \ldots, r'(\mathbf{g}_i)\, \mathbf{x}_j, \ldots, 0\right]^\top \\ &= \left((\mathbf{z} - \mathbf{y})^\top W\right)_i r'(\mathbf{g}_i)\, \mathbf{x}_j. \end{aligned}$$

Q2. [9 pts] A Miscellany

(a) [3 pts] Consider a convolutional neural network for reading the handwritten MNIST letters, which are 28×28 images. Suppose the first hidden layer is a convolutional layer with 20 different 5×5 filters, applied to the input image with a stride of 1 (i.e., every filter is applied to *every* 5×5 patch of the image, with patches allowed to overlap). Each filter has a bias weight. How many weights (parameters) does this layer use?

$20 \times (5 \times 5 + 1) = 520$.

(b) [3 pts] Let X be an $n \times d$ design matrix representing n sample points in $\mathbb{R}^d$. Let $X = UDV^\top$ be the singular value decomposition of X. We stated in lecture that row i of the matrix UD gives the coordinates of sample point X_i in principal coordinates space, i.e., $X_i \cdot v_j$ for each j, where X_i is the ith row of X and v_j is the jth column of V. Show that this is true.

As V is an orthogonal matrix, we can write $XV = UDV^\top V = UD$.

By the definition of matrix multiplication, $(UD)_{ij} = X_i \cdot v_j$.

(c) [3 pts] Let $x, y \in \mathbb{R}^d$ be two points (e.g., sample or test points). Consider the function $k(x, y) = x^\top \text{rev}(y)$ where $\text{rev}(y)$ reverses the order of the components in y. For example, $\text{rev}\left(\begin{bmatrix}1\\2\\3\end{bmatrix}\right) = \begin{bmatrix}3\\2\\1\end{bmatrix}$. Show that k *cannot* be a valid kernel function.

Hint: remember how the kernel function is defined, and show a simple two-dimensional counterexample.

We have that $k((-1, 1), (-1, 1)) = -2$, but this is impossible as, if k is a valid kernel, then there is some function Φ such that $k(x, x) = \Phi(x)^\top \Phi(x) \geq 0$.

Q3. [10 pts] Maximum Likelihood Estimation for Reliability Testing

Suppose we are reliability testing n units taken randomly from a population of identical appliances. We want to estimate the mean failure time of the population. We assume the failure times come from an exponential distribution with parameter $\lambda > 0$, whose probability density function is $f(x) = \lambda e^{-\lambda x}$ (on the domain $x \geq 0$) and whose cumulative distribution function is $F(x) = \int_0^x f(x)\,\mathrm{d}x = 1 - e^{-\lambda x}$.

(a) [6 pts] In an ideal (but impractical) scenario, we run the units until they all fail. The failure times are $t_1, t_2, \ldots, t_n$.

Formulate the likelihood function $\mathcal{L}(\lambda; t_1, \ldots, t_n)$ for our data. Then find the maximum likelihood estimate $\hat{\lambda}$ for the distribution's parameter.

$$\begin{aligned}
\mathcal{L}(\lambda; t_1, \ldots, t_n) &= \prod_{i=1}^{n} f(t_i) = \prod_{i=1}^{n} \lambda e^{-\lambda t_i} = \lambda^n e^{-\lambda \sum_{i=1}^{n} t_i} \\
\ln \mathcal{L}(\lambda) &= n \ln \lambda - \lambda \sum_{i=1}^{n} t_i \\
\frac{\partial}{\partial \lambda} \ln \mathcal{L}(\lambda) &= \frac{n}{\lambda} - \sum_{i=1}^{n} t_i = 0 \\
\hat{\lambda} &= \frac{n}{\sum_{i=1}^{n} t_i}
\end{aligned}$$

(b) [4 pts] In a more realistic scenario, we run the units for a fixed time T. We observe r unit failures, where $0 \leq r \leq n$, and there are $n - r$ units that survive the entire time T without failing. The failure times are $t_1, t_2, \ldots, t_r$.

Formulate the likelihood function $\mathcal{L}(\lambda; n, r, t_1, \ldots, t_r)$ for our data. Then find the maximum likelihood estimate $\hat{\lambda}$ for the distribution's parameter.

Hint 1: What is the probability that a unit will not fail during time T? *Hint 2:* It is okay to define $\mathcal{L}(\lambda)$ in a way that includes contributions (densities and probability masses) that are not commensurate with each other. Then the constant of proportionality of $\mathcal{L}(\lambda)$ is meaningless, but that constant is irrelevant for finding the best-fit parameter $\hat{\lambda}$. *Hint 3:* If you're confused, for part marks write down the likelihood that r units fail and $n-r$ units survive; then try the full problem. *Hint 4:* If you do it right, $\hat{\lambda}$ will be the number of observed failures divided by the sum of unit test times.

$$\begin{aligned}
\mathcal{L}(\lambda; n, r, t_1, \ldots, t_r) &\propto \left(\prod_{i=1}^{r} f(t_i) \right) (1 - F(T))^{n-r} \\
&= \left(\prod_{i=1}^{r} \lambda e^{-\lambda t_i} \right) \left(e^{-\lambda T} \right)^{n-r} \\
&= \lambda^r e^{-\lambda \sum_{i=1}^{r} t_i} e^{-\lambda (n-r) T} \\
\ln \mathcal{L}(\lambda) &= r \ln \lambda - \lambda \sum_{i=1}^{r} t_i - \lambda (n-r) T + \text{constant} \\
\frac{\partial}{\partial \lambda} \ln \mathcal{L}(\lambda) &= \frac{r}{\lambda} - \sum_{i=1}^{r} t_i - (n-r) T = 0 \\
\hat{\lambda} &= \frac{r}{\sum_{i=1}^{r} t_i + (n-r) T}
\end{aligned}$$

Q4. [10 pts] Decision Trees

Consider the design matrix

$$\begin{bmatrix} 4 & 6 & 9 & 1 & 7 & 5 \\ 1 & 6 & 5 & 2 & 3 & 4 \end{bmatrix}^\top$$

representing 6 sample points, each with two features f_1 and f_2.

The labels for the data are

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}^\top$$

In this question, we build a decision tree of depth 2 by hand to classify the data.

(a) [2 pts] What is the entropy at the root of the tree?

$$-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

(b) [3 pts] What is the rule for the first split? Write your answer in a form like $f_1 \geq 4$ or $f_2 \geq 3$. Hint: you should be able to eyeball the best split without calculating the entropies.

If we sort by f_1, the features and the corresponding labels are

$$\begin{bmatrix} 1 & 4 & 5 & 6 & 7 & 9 \\ 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}.$$

If we sort by f_2, we have

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

The best split is $f_1 \geq 7$.

(c) [3 pts] For each of the two treenodes after the first split, what is the rule for the second split?

For the treenode with labels $(1, 1)$, there's no need to split again.

For the treenode with labels $(0, 1, 0, 0)$, if we sort by f_1, we have

$$\begin{bmatrix} 1 & 4 & 5 & 6 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

If we sort using f_2, we get

$$\begin{bmatrix} 1 & 2 & 4 & 6 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

We easily see we should choose $f_2 \geq 2$.

(d) [2 pts] Let's return to the root of the tree, and suppose we're incompetent tree builders. Is there a (not trivial) split at the root that would have given us an information gain of zero? Explain your answer.

Yes. The rules $f_1 \geq 5$, $f_2 \geq 3$, or $f_2 \geq 5$ would all fail to reduce the weighted average entropy below 1.

Q5. [11 pts] Bagging and Random Forests

We are building a random forest for a 2-class classification problem with t decision trees and bagging. The input is a $n \times d$ design matrix X representing n sample points in $\mathbb{R}^d$ (quantitative real-valued features only). For the ith decision tree we create an n-point training set $X^{(i)}$ through standard bagging. At each node of each tree, we randomly select k of the features (this random subset is selected independently for each treenode) and choose the single-feature split that maximizes the information gain, compared to all possible single-feature splits on those k features. Assume that we can radix sort real numbers in linear time, and we can randomly select an item from a set in constant time.

(a) [3 pts] Remind us how bagging works. How do we generate the data sets $X^{(i)}$? What do we do with duplicate points?

For each training set $X^{(i)}$, we select n sample points from X uniformly at random **with replacement**. Duplicate points have proportionally greater weight in the entropy (or other cost function) calculations. (We will accept an answer that states that duplicate points are treated as if they were separate points infinitesimally close together.)

(b) [3 pts] Fill in the blanks to derive the overall running time to construct a random forest with bagging and random subset selection. Let h be the height/depth (they're the same thing) of the tallest/deepest tree in the forest. You must use the tightest bounds possible with respect to n, d, t, k, h, and n'.

Consider choosing the split at a treenode whose box contains n' sample points. We can choose the best split for these n' sample points in $O($_____$)$ time. Therefore, the running time **per sample point in that node** is $O($_____$)$.

Each sample point in $X^{(i)}$ participates in at most $O($_____$)$ treenodes, so each sample point contributes at most $O($_____$)$ to the time. Therefore, the total running time for one tree is $O($_____$)$.

We have t trees, so the total running time to create the random forest is $O($_____$)$.

The blanks in order: $O(n'k)$, $O(k)$, $O(h)$, $O(kh)$, $O(nkh)$, $O(nkht)$. They are each worth half a point.

(c) [2 pts] If we instead use a support vector machine to choose the split in each treenode, how does that change the asymptotic **query** time to classify a test point?

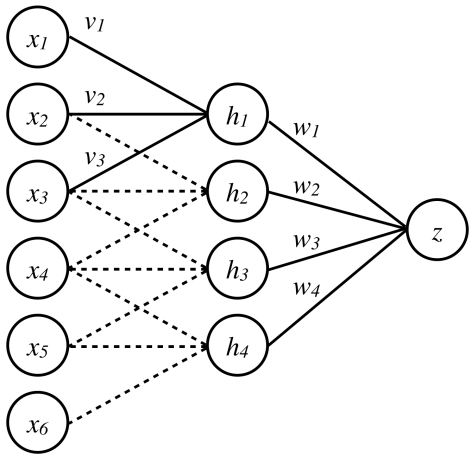
It slows queries down by a factor of $\Theta(d)$ or $\Theta(k)$ (depending whether you run the SVM on k features or all d features—we'll accept either interpretation), because it is necessary to inspect all d (or k) features of the query point at each treenode.

(d) [3 pts] Why does bagging by itself (without random subset selection) tend **not** to improve the performance of decision trees as much as we might expect?

It is common that the same few features tend to dominate in all of the subsets, so almost all the trees will tend to have very similar early splits, and therefore all the trees will produce very similar estimates. The models are not decorrelated enough.

Q6. [11 pts] One-Dimensional ConvNet Backprop

Consider this convolutional neural network architecture.



In the first layer, we have a one-dimensional convolution with a single filter of size 3 such that $h_i = s\left(\sum_{j=1}^{3} v_j x_{i+j-1}\right)$. The second layer is fully connected, such that $z = \sum_{i=1}^{4} w_i h_i$. The hidden units' activation function $s(x)$ is the logistic (sigmoid) function with derivative $s'(x) = s(x)\,(1 - s(x))$. The output unit is linear (no activation function). We perform gradient descent on the loss function $R = (y - z)^2$, where y is the training label for x.

(a) [1 pt] What is the total number of parameters in this neural network? Recall that convolutional layers share weights. There are no bias terms.

The answer is 7. There are 3 parameters in layer 1 and 4 parameters in layer 2.

(b) [4 pts] Compute $\partial R / \partial w_i$.

$$\frac{\partial R}{\partial w_i} = -2(y - z)h_i$$

(c) [1 pt] Vectorize the previous expression—that is, write $\partial R / \partial w$.

$$\frac{\partial R}{\partial w} = -2(y - z)h$$

(d) [5 pts] Compute $\partial R / \partial v_j$.

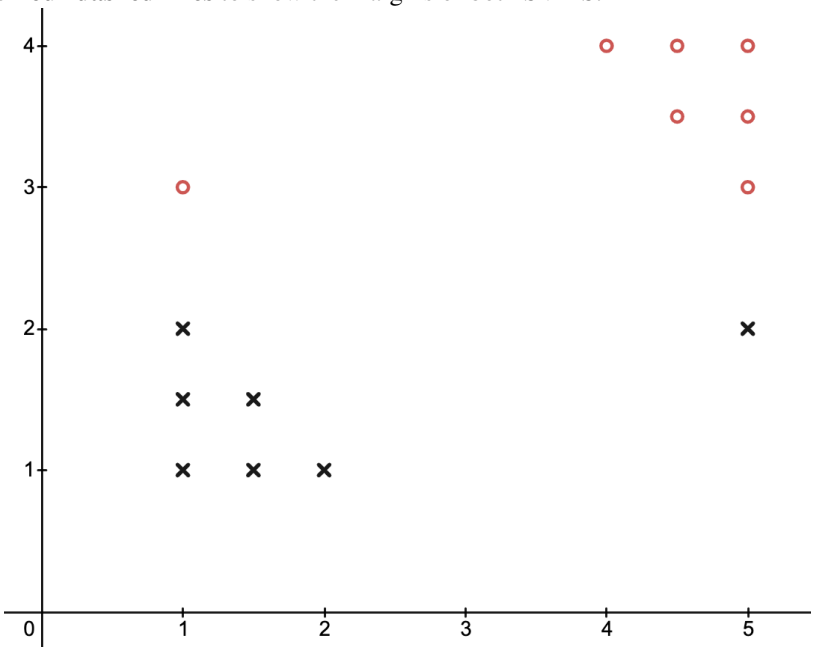
$$\frac{\partial R}{\partial v_j} = -2(y - z)\frac{\partial z}{\partial v_j} = -2(y - z)\sum_{i=1}^{4}\frac{\partial z}{\partial h_i}\frac{\partial h_i}{\partial v_j} = -2(y - z)\sum_{i=1}^{4} w_i h_i (1 - h_i) x_{i+j-1}$$

Q3. [10 pts] Decision Boundaries

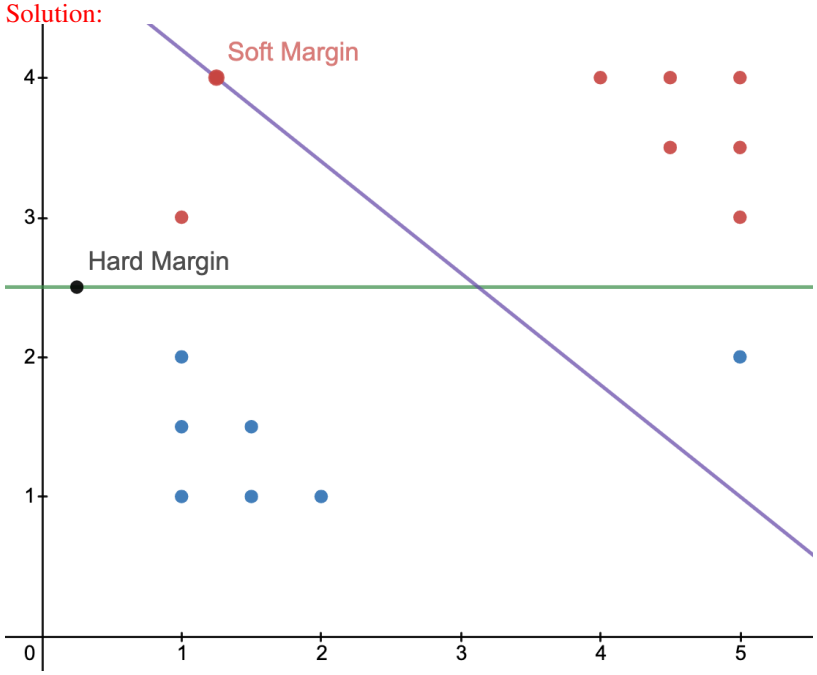
In the question, you will draw the decision boundaries that classifiers would learn.

(a) [6 pts] Given the sample points below, draw and label **two lines**: the decision boundary learned by a hard-margin SVM and the decision boundary learned by a soft-margin SVM. We are not specifying the hyperparameter C, but don't make C too extreme. (We are looking for a qualitative difference between hard- and soft-margin SVMs.) **Label the two lines clearly.**

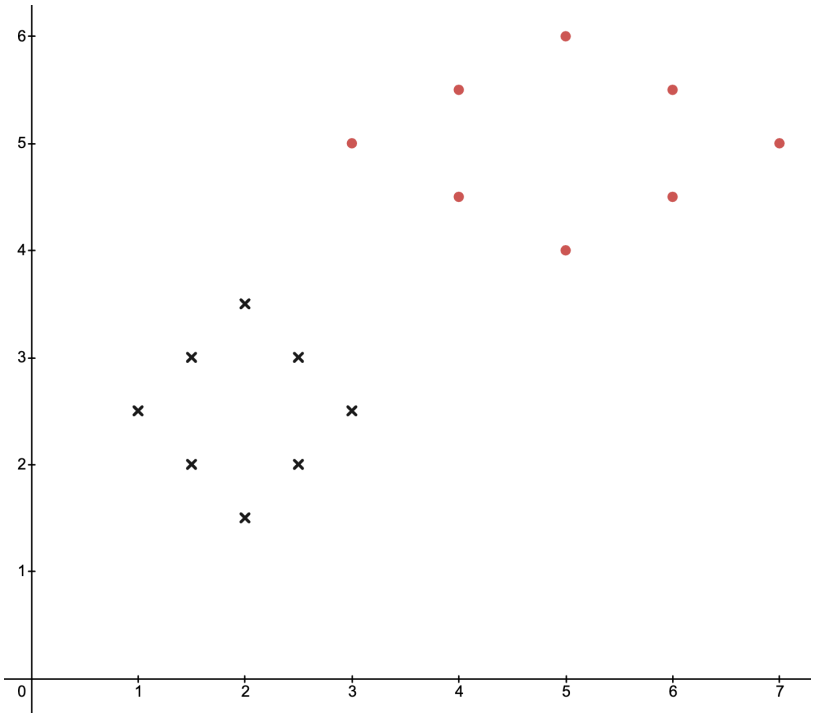
Also draw and label **four dashed lines** to show the margins of both SVMS.



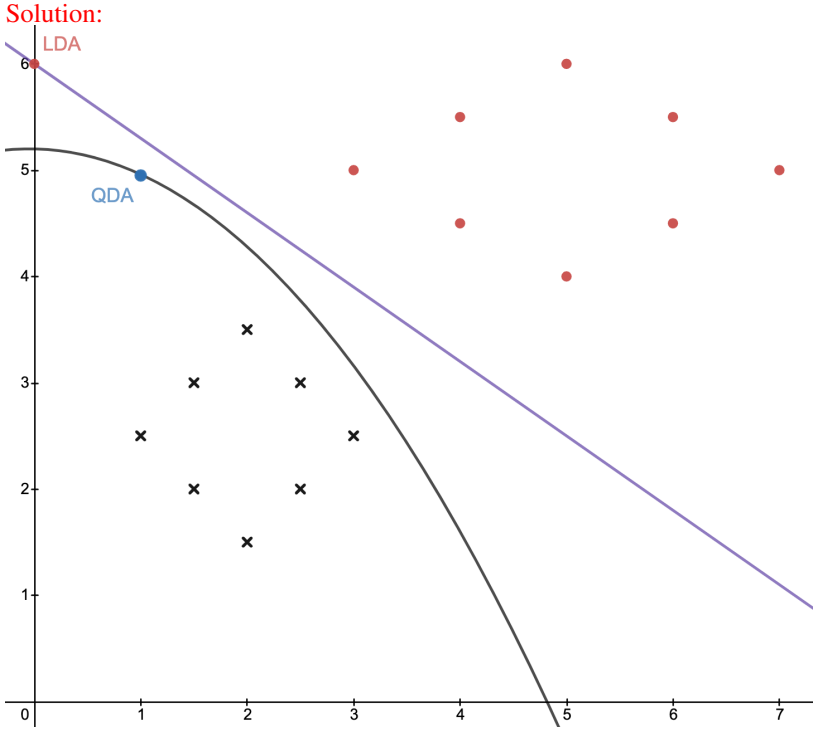
Solution:



(b) [4 pts] Given the sample points below, draw and label **two curves**: the decision boundary learned by LDA and the decision boundary learned by QDA. Label the two curves clearly.



Solution:



Q4. [16 pts] Kernel Principal Components Analysis

Let X be an $n \times d$ design matrix. Suppose that X has been centered, so the sample points in X have mean zero. In this problem we consider kernel PCA and show that it equates to solving a *generalized Rayleigh quotient problem*.

(a) [1 pt] Fill in the blank: every principal component direction for X is an eigenvector of _____. $X^\top X$

(b) [1 pt] Fill in the blank: an optimization problem can be kernelized only if its solution w is always a linear combination of the sample points. In other words, we can write it in the form $w =$ _____.
$X^\top a$ (for some vector a).

(c) [4 pts] Show that every principal component direction w with a nonzero eigenvalue is a linear combination of the sample points (even when $n < d$).

As w is an eigenvector of $X^\top X$, there is a $\lambda \in \mathbb{R}$ such that $X^\top X w = \lambda w$. Setting $a = \frac{1}{\lambda} X w$, we have $X^\top a = w$.

(d) [4 pts] Let $\Phi(z)$ be a feature map that takes a point $z \in \mathbb{R}^d$ and maps it to a point $\Phi(z) \in \mathbb{R}^D$, where D might be extremely large or even infinite. But suppose that we can compute the kernel function $k(x, z) = \Phi(x) \cdot \Phi(z)$ much more quickly than we can compute $\Phi(x)$ directly. Let $\Phi(X)$ be the $n \times D$ matrix in which each sample point is replaced by a featurized point. By our usual convention, row i of X is $X_i^\top$, and row i of $\Phi(X)$ is $\Phi(X_i)^\top$.

Remind us: what is the kernel matrix K? Answer this two ways: explain the relationship between K and the kernel function $k(\cdot, \cdot)$; then write the relationship between K and $\Phi(X)$. Lastly, show that these two definitions are equivalent.

K is the $n \times n$ matrix with components $K_{ij} = k(X_i, X_j)$. Also, $K = \Phi(X)\Phi(X)^\top$.

These two characterizations are equivalent because $K_{ij} = k(X_i, X_j) = \Phi(X_i) \cdot \Phi(X_j)$ is the inner product of row i of $\Phi(X)$ and column j of $\Phi(X)^\top$, which implies that $K = \Phi(X)\Phi(X)^\top$.

(e) [2 pts] Fill in the space: the first principle component direction of the *featurized* design matrix $\Phi(X)$ is any nonzero vector $w \in \mathbb{R}^D$ that maximizes the *Rayleigh quotient*, which is .

$$\frac{w^\top \Phi(X)^\top \Phi(X) w}{w^\top w}.$$

(f) [4 pts] Show that the problem of maximizing this Rayleigh quotient is equivalent to maximizing

$$\frac{a^\top B a}{a^\top C a}$$

for some positive semidefinite matrices $B, C \in \mathbb{R}^{n \times n}$, where $a \in \mathbb{R}^n$ is a vector of dual weights. This expression is called a *generalized Rayleigh quotient*. What are the matrices B and C? For full points, express them in a form that does not require any direct computation of the feature vectors Φ, which could be extremely long.

$\frac{w^\top \Phi(X)^\top \Phi(X) w}{w^\top w} = \frac{a^\top \Phi(X)\Phi(X)^\top \Phi(X)\Phi(X)^\top a}{a^\top \Phi(X)\Phi(X)^\top a} = \frac{a^\top K^2 a}{a^\top K a}$. Hence $B = K^2$ and $C = K$.

Q6. [17 pts] Learning Mixtures of Gaussians with k-Means

Let $X_1, \ldots, X_n \in \mathbb{R}^d$ be independent, identically distributed points sampled from a mixture of two normal (Gaussian) distributions. They are drawn independently from the probability distribution function (PDF)

$$p(x) = \theta\, N_1(x) + (1-\theta)\, N_2(x), \quad \text{where } N_1(x) = \frac{1}{(\sqrt{2\pi})^d} e^{-\|x-\mu_1\|^2/2} \text{ and } N_2(x) = \frac{1}{(\sqrt{2\pi})^d} e^{-\|x-\mu_2\|^2/2}$$

are the PDFs for the isotropic multivariate normal distributions $\mathcal{N}(\mu_1, 1)$ and $\mathcal{N}(\mu_2, 1)$, respectively. The parameter $\theta \in (0, 1)$ is called the *mixture proportion*. In essence, we flip a biased coin to decide whether to draw a point from the first Gaussian (with probability θ) or the second (with probability $1 - \theta$).

Each data point is generated as follows. First draw a random Z_i, which has value 1 with probability θ, and has value 2 with probability $1 - \theta$. Then, draw $X_i \sim \mathcal{N}(\mu_{Z_i}, 1)$. Our learning algorithm gets X_i as an input, but does not know Z_i.

Our goal is to find the maximum likelihood estimates of the three unknown distribution parameters $\theta \in (0, 1)$, $\mu_1 \in \mathbb{R}^d$, and $\mu_2 \in \mathbb{R}^d$ from the sample points $X_1, \ldots, X_n$. Unlike MLE for one Gaussian, it is **not** possible to give explicit analytic formulas for these estimates. Instead, we develop a variant of k-means clustering which (often) converges to the correct maximum likelihood estimates of θ, μ_1, and μ_2. This variant doesn't assign each point entirely to one cluster; rather, each point is assigned an estimated posterior probability of coming from normal distribution 1.

(a) [4 pts] Let $\tau_i = P(Z_i = 1 | X_i)$. That is, τ_i is the posterior probability that point X_i has $Z_i = 1$. Use Bayes' Theorem to express τ_i in terms of X_i, θ, μ_1, μ_2, and the Gaussian PDFs $N_1(x)$ and $N_2(x)$. To help you with part (c), also write down a similar formula for $1 - \tau_i$, which is the posterior probability that $Z_i = 2$.

Bayes' Theorem implies that

$$\tau_i = \frac{\theta N_1(X_i)}{\theta N_1(X_i) + (1-\theta) N_2(X_i)}, \quad 1 - \tau_i = \frac{(1-\theta) N_2(X_i)}{\theta N_1(X_i) + (1-\theta) N_2(X_i)}.$$

(b) [3 pts] Write down the log-likelihood function, $\ell(\theta, \mu_1, \mu_2; X_1, \ldots, X_n) = \ln p(X_1, \ldots, X_n)$, as a summation. Note: it doesn't simplify much.

Because the samples are iid, $p(X_1, \ldots, X_n) = \prod_{i=1}^n p(X_i)$, so

$$\ell(\theta, \mu_1, \mu_2; X_1, \ldots, X_n) = \sum_{i=1}^n \ln(\theta N_1(X_i) + (1-\theta) N_2(X_i)).$$

(c) [3 pts] Express $\frac{\partial \ell}{\partial \theta}$ in terms of θ and $\tau_i, i \in \{1, \ldots, n\}$ and simplify as much as possible. There should be no normal PDFs explicitly in your solution, though the τ_i's may implicitly use them. Hint: Recall that $(\ln f(x))' = \frac{f'(x)}{f(x)}$.

$$\frac{\partial \ell}{\partial \theta} = \sum_{i=1}^n \left(\frac{\tau_i}{\theta} - \frac{1-\tau_i}{1-\theta} \right) = \frac{1}{\theta - \theta^2} \sum_{i=1}^n (\tau_i - \theta) = \frac{(\sum \tau_i) - \theta n}{\theta - \theta^2}.$$

(Any of these expressions is simple enough to receive full marks.)

(d) [4 pts] Express $\nabla_{\mu_1}\ell$ in terms of μ_1 and $\tau_i, X_i, i \in \{1,\ldots,n\}$. Do the same for $\nabla_{\mu_2}\ell$ (but in terms of μ_2 rather than μ_1). Again, there should be no normal PDFs explicitly in your solution, though the τ_i's may implicitly use them.
Hint: It will help (and get you part marks) to first write $\nabla_{\mu_1} N_1(x)$ as a function of $N_1(x)$, x, and μ_1.

$$\begin{aligned}\nabla_{\mu_1}\ell &= \sum_{i=1}^{n} \frac{\theta \nabla_{\mu_1} N_1(X_i)}{\theta N_1(X_i) + (1-\theta)N_2(X_i)} \\ &= \sum_{i=1}^{n} \frac{\theta N_1(X_i)}{\theta N_1(X_i) + (1-\theta)N_2(X_i)}(X_i - \mu_1) \\ &= \sum_{i=1}^{n} \tau_i (X_i - \mu_1).\end{aligned}$$

Similarly,

$$\nabla_{\mu_2}\ell = \sum_{i=1}^{n} (1-\tau_i)(X_i - \mu_2).$$

(e) [3 pts] We conclude: if we know μ_1, μ_2, and θ, we can compute the posteriors τ_i. On the other hand, if we know the τ_i's, we can estimate μ_1, μ_2, and θ by using the derivatives in parts (c) and (d) to find the maximum likelihood estimates. This leads to the following k-means-like algorithm.

- Initialize $\tau_1, \tau_2, \ldots, \tau_n$ to arbitrary values in the range $[0, 1]$.
- Repeat the following two steps.
  1. Update the Gaussian cluster parameters: for fixed values of $\tau_1, \tau_2, \ldots, \tau_n$, update μ_1, μ_2, and θ.
  2. Update the posterior probabilities: for fixed values of μ_1, μ_2 and θ, update $\tau_1, \tau_2, \ldots, \tau_n$.

In part (a), you wrote the update rule for step 2. Using your results from parts (c) and (d), write down the explicit update formulas for step 1.

$$\mu_1 \leftarrow \frac{\sum_{i=1}^{n} \tau_i X_i}{\sum_{i=1}^{n} \tau_i}, \quad \mu_2 \leftarrow \frac{\sum_{i=1}^{n} (1-\tau_i) X_i}{\sum_{i=1}^{n} (1-\tau_i)}, \quad \theta \leftarrow \frac{1}{n}\sum_{i=1}^{n} \tau_i.$$

Q2. [14 pts] Principal Components Analysis

Consider the following design matrix, representing four sample points $X_i \in \mathbb{R}^2$.

$$X = \begin{bmatrix} 4 & 1 \\ 2 & 3 \\ 5 & 4 \\ 1 & 0 \end{bmatrix}.$$

We want to represent the data in only one dimension, so we turn to principal components analysis (PCA).

(1) [5 pts] Compute the **unit-length principal component directions** of X, and **state which one the PCA algorithm would choose** if you request just one principal component. Please provide an exact answer, without approximation. (You will need to use the square root symbol.) **Show your work!**

We center X, yielding

$$\dot{X} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 2 & 2 \\ -2 & -2 \end{bmatrix}.$$

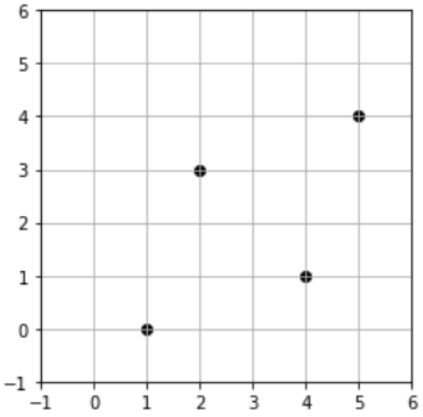
Then $\dot{X}^\top \dot{X} = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}$. (Divide by 4 if you want the sample covariance matrix. But we don't care about the magnitude.)

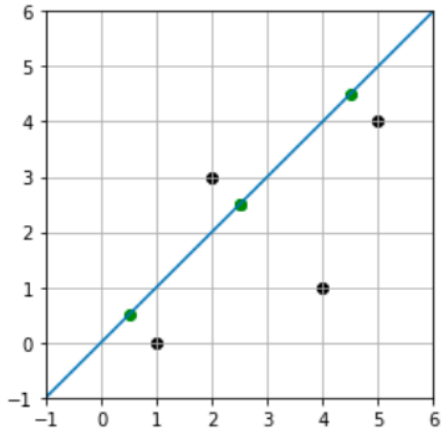
Its eigenvectors are $[1/\sqrt{2} \quad 1/\sqrt{2}]^\top$ with eigenvalue 16 and $[1/\sqrt{2} \quad -1/\sqrt{2}]^\top$ with eigenvalue 4. The former eigenvector is chosen.

(Negated versions of these vectors also get full points.)

(2) [5 pts] The plot below depicts the sample points from X. We want a one-dimensional representation of the data, so **draw the principal component direction (as a line) and the projections of all four sample points onto the principal direction**.

Label each projected point with its principal coordinate value (where the origin's principal coordinate is zero). Give the principal coordinate values exactly.



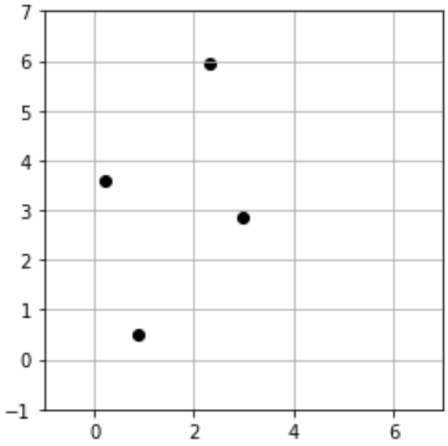


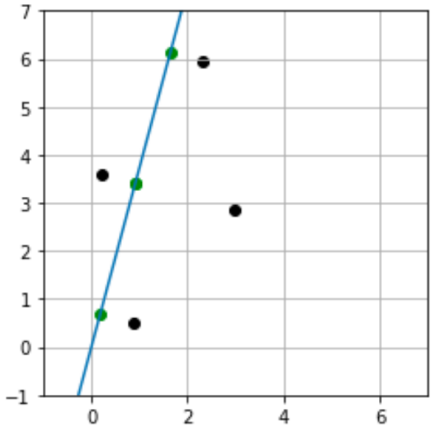
The principal coordinates are $\frac{1}{\sqrt{2}}$, $\frac{5}{\sqrt{2}}$, $\frac{5}{\sqrt{2}}$, and $\frac{9}{\sqrt{2}}$. (Alternatively, all of these could be negative, but they all have to have the same sign.)

(3) [4 pts] The plot below depicts the sample points from X rotated 30 degrees counterclockwise about the origin.

As in part (b), **identify the principal component direction that the PCA algorithm would choose and draw it (as a line) on the plot**. Also **draw the projections of the rotated points** onto the principal direction.

Label each projected point with the exact value of its principal coordinate.





The line passes through the origin and is parallel to the two sample points that are farthest apart, so it's easy to draw. Rotation has not changed the principal coordinates: $\frac{1}{\sqrt{2}}$, $\frac{5}{\sqrt{2}}$, $\frac{5}{\sqrt{2}}$, and $\frac{9}{\sqrt{2}}$. (Again, these could all be negative.)

Q3. [14 pts] A Decision Tree

In this question we investigate whether students will pass or fail CS 189 based on whether or not they studied, cheated, and slept well before the exam. You are given the following data for five students. There are three features, "Studied," "Slept," and "Cheated." The column "Result" shows the label we want to predict.

	Studied	**Slept**	**Cheated**	**Result**
Student 1	Yes	No	No	Passed
Student 2	Yes	No	Yes	Failed
Student 3	No	Yes	No	Failed
Student 4	Yes	Yes	Yes	Failed
Student 5	Yes	Yes	No	Passed

(1) [4 pts] What is the **entropy** $H(\text{Result})$ at the root node? (There is no need to compute the exact number; you may write it as an arithmetic expression.)

$$H(\text{Result}) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right).$$

(2) [5 pts] **Draw the decision tree** where every split maximizes the information gain. (An actual drawing, please; a written description does not suffice.) Do not perform a split on a pure leaf or if the split will produce an empty child; otherwise, split. **Explain (with numbers)** why you chose the splits you chose.

A tree that first splits on "Cheated" and then "Studied."

(3) [2 pts] Did the tree you built implicitly perform feature subset selection? **Explain.**

Yes, because it does not use the feature "Slept."

(4) [3 pts] Suppose you have a sample of n students for some large n, with the same three features. Assuming that we use a reasonably efficient algorithm to build the tree (as discussed in class), what is the **worst-case running time to build** the decision tree? (Write your answer in the simplest asymptotic form possible.) **Why?**

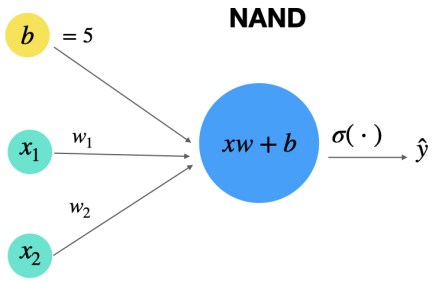
We have 3 binary features, so the tree's depth cannot exceed 3 and each sample point participates in at most four treenodes. Hence, it cannot take more than $\Theta(n)$ time to build the tree.

Q6. [10 pts] A Miscellany

(1) [4 pts] Consider a single unit in a neural network that receives two binary inputs $x_1, x_2 \in \{0, 1\}^2$ and computes a linear combination followed by a threshold activation function, namely,

$$\sigma(z) = \begin{cases} 1, & z \geq 0, \\ 0 & \text{otherwise.} \end{cases}$$

The unit is illustrated below. We have chosen a bias term of $b = 5$. **Provide values for the two weights w_1 and w_2** that allow you to compute the NAND function (which is 0 if and only if both inputs are 1).



$w_1 = -3$, $w_2 = -3$ will work.

(2) [6 pts] We are drawing sample points from a distribution with the probability density function (PDF) $f(x) = \frac{1}{2}\, e^{-|x-\mu|}$, but we do not know the mean $\mu \in \mathbb{R}$. We decide to estimate μ with maximum likelihood estimation (MLE). Unfortunately, we have only two sample points $X_1, X_2 \in \mathbb{R}$.

Derive the likelihood and the log-likelihood for this problem. Then **show that every value of μ between X_1 and X_2 is a maximum likelihood estimate**.

The likelihood is

$$\mathcal{L}(\mu; X_1, X_2) = \frac{1}{4} e^{-|X_1-\mu|} e^{-|X_2-\mu|},$$

and the log-likelihood is

$$\ell(\mu; X_1, X_2) = -|X_1 - \mu| - |X_2 - \mu| - \ln 4.$$

For any μ between X_1 and X_2 (inclusive), the log-likelihood is $-|X_1 - X_2| - \ln 4$. For any μ outside that range, it is lesser. (For example if μ is less than $\min\{X_1, X_2\}$, then the log-likelihood is $-|X_1 - X_2| - 2|\mu - \min\{X_1, X_2\}| - \ln 4$).

Q3. [20 pts] The Singular Value Decomposition

We want you to compute (by hand) part of the singular value decomposition (SVD) $X = UDV^\top$ of the matrix

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{bmatrix}.$$

(a) [6 pts] One way to compute the SVD by hand is to exploit the relationship between the singular values of X and the eigenvalues of a related symmetric matrix. What symmetric matrix has eigenvalues related to the singular values of X? Compute that matrix, write down its characteristic polynomial and simplify it, and determine its eigenvalues. Show your work!

(b) [4 pts] What is the relationship between the singular values of X and the eigenvalues of your symmetric matrix? Use that relationship to write down the two specific singular values of X.

(c) [5 pts] Generalize your answer above. For **any** matrix Z (not only X), what symmetric matrix has eigenvalues related to the singular values of Z? **Prove** that every singular value of Z can be identified by looking at the eigenvalues of the matrix your method constructs.

(d) [5 pts] Explicitly write down enough eigenvectors of your symmetric matrix to form a complete basis for the space. (Hint: the symmetry of your matrix should make it easy to guess eigenvectors; then you can confirm that they are eigenvectors.) Based on these, write down the value of U or V (whichever is appropriate).

(a) Consider the symmetric matrix

$$Y = X^\top X = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

The characteristic polynomial of Y is $(\lambda - 2)^2 - 1 = \lambda^2 - 4\lambda + 3$. We can factor it into $(\lambda - 3)(\lambda - 1)$ or simply use the quadratic formula to see that its solutions are $\lambda = 3$ and $\lambda = 1$, the eigenvalues of Y.

Note: if you like doing things the hard way, it is also correct to use the matrix

$$XX^\top = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ -1 & -1 & 2 \end{bmatrix}.$$

(b) The singular values of X are square roots of eigenvalues of $X^\top X$ (or $XX^\top$). Therefore, they are $\sqrt{3}$ and 1.

(c) The singular values of Z are square roots of eigenvalues of $Z^\top Z$ (or $ZZ^\top$). Proof: let $Z = UDV^\top$ be the SVD of Z; then $Z^\top Z = VDU^\top UDV^\top = VD^2V^\top$, which is clearly an eigendecomposition (because V is orthonormal and D^2 is diagonal). If Z has a singular vector δ with right singular vector v, then v is an eigenvalue of $Z^\top Z$ with eigenvalue δ^2.

(d) The simplest basis of eigenvectors for $X^\top X$ is $[1, 1]^\top$ and $[1, -1]^\top$. From this we can choose

$$V = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix},$$

but note that the $-1/\sqrt{2}$ could appear in any of the four slots. Moreover, we could have three negative entries instead of one.

Postscript: By the way, one correct SVD is

$$X = \begin{bmatrix} 1/\sqrt{6} & -1/\sqrt{2} \\ 1/\sqrt{6} & 1/\sqrt{2} \\ 2/\sqrt{6} & 0 \end{bmatrix} \begin{bmatrix} \sqrt{3} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}.$$

Q4. [10 pts] Maximum Likelihood Estimation of a Mean

In class, we derived the maximum likelihood estimates for the mean and variance of a univariate (or isotropic multivariate) normal distribution, based on n points sampled independently from that distribution. Then we considered the more general (anisotropic) multivariate distribution $\mathcal{N}(\mu, \Sigma)$, and I told you the MLE estimates without deriving them. Let's derive the maximum likelihood estimate for the mean.

We are given n independent sample points $X_1, X_2, \ldots, X_n \sim \mathcal{N}(\mu, \Sigma)$ with d features each, where $\mu \in \mathbb{R}^d$ and $\Sigma \in \mathbb{R}^{d \times d}$ is symmetric and positive definite.

(a) [4 pts] Write out an expression for the likelihood $\mathcal{L}(\mu, \Sigma; X_1, \ldots, X_n)$. (You should substitute in the normal PDF; a placeholder will only get part marks.) Simplify the expression as much as possible.

(b) [6 pts] Derive the maximum likelihood estimate $\hat{\mu}$ of the distribution mean μ. (You probably already know what its value should be. If you get it right, it will not depend on Σ.)

(a)

$$\mathcal{L}(\mu, \Sigma) = \frac{1}{(2\pi)^{nd/2}|\Sigma|^{n/2}} \prod_{i=1}^{n} \exp\left(-\frac{1}{2}(X_i - \mu)^\top \Sigma^{-1}(X_i - \mu)\right).$$

Alternatively,

$$\mathcal{L}(\mu, \Sigma) = \frac{1}{(2\pi)^{nd/2}|\Sigma|^{n/2}} \exp\left(-\frac{1}{2}\sum_{i=1}^{n}(X_i - \mu)^\top \Sigma^{-1}(X_i - \mu)\right).$$

(b) The log-likelihood is

$$\ell(\mu, \Sigma) = \ln \mathcal{L}(\mu, \Sigma) = -\ln\left((2\pi)^{nd/2}|\Sigma|^{n/2}\right) - \frac{1}{2}\sum_{i=1}^{n}(X_i - \mu)^\top \Sigma^{-1}(X_i - \mu).$$

Setting $\nabla_\mu \ell(\mu, \Sigma) = 0$ gives (because Σ^{-1} is symmetric)

$$\Sigma^{-1}\left(\sum_{i=1}^{n} X_i - n\mu\right) = 0.$$

Pre-multiplying both sides by Σ gives $\sum_{i=1}^{n} X_i - n\mu = 0$, hence $\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} X_i$, which is the sample mean.

Q5. [20 pts] Kernel Principal Components Analysis

In this problem, we will derive the kernelized version of principal components analysis (PCA). You are given a **centered** $n \times d$ design matrix X whose rows express the sample points $X_1^\top, X_2^\top, ..., X_n^\top$, and a feature map Φ that maps each sample point $X_i \in \mathbb{R}^d$ to a high-dimensional "lifted" sample point $\Phi(X_i) \in \mathbb{R}^D$, where $D \gg d$. Define the kernel function $k(X_i, X_j) = \Phi(X_i)^\top \Phi(X_j)$, and suppose it takes $O(d)$ time to compute (which is much, much less than D).

The sample covariance matrix of the lifted points in $\mathbb{R}^D$ is $C = \frac{1}{n}\Phi(X)^\top\Phi(X) = \frac{1}{n}\sum_{i=1}^n \Phi(X_i)\,\Phi(X_i)^\top \in \mathbb{R}^{D\times D}$. (Recall that $\Phi(X) \in \mathbb{R}^{n\times D}$. Recall also that our convention is to express each X_i and each $\Phi(X_i)$ as a column vector, but these points are stored as rows of X and $\Phi(X)$, respectively, which is why the transpose moved.)

(a) [6 pts] To perform PCA, we want to find the eigenvectors of C with nonzero eigenvalues. However, we don't ever want to construct C—it's way too big! We don't even want to construct one eigenvector of C! Show that if a vector $v \in \mathbb{R}^D$ is an eigenvector of C with a nonzero eigenvalue, then v can be expressed as a linear combination of lifted sample points: that is, $v = \sum_{j=1}^n a_j \Phi(X_j) = \Phi(X)^\top a$ for some vector $a \in \mathbb{R}^n$ of "dual" coefficients. Hint: think of the standard definition of an eigenvector of C, then substitute in the value of C.

(b) [5 pts] Recall that the kernel matrix $K = \Phi(X)\,\Phi(X)^\top$ is the symmetric $n \times n$ matrix such that $K_{ij} = k(X_i, X_j)$. In the circumstance described in part (a), prove that $K^2 a = \lambda n K a$. Hint: again start with the standard definition of an eigenvector of C, then substitute in the value of v from above.

(c) [5 pts] Conversely, suppose that a is an eigenvector of K with eigenvalue λ_a. Show that $v = \Phi(X)^\top a$ is an eigenvector of C. What is v's corresponding eigenvalue?

(d) [4 pts] The parts above imply that by finding the eigendecomposition of the kernel matrix K, we can indirectly express the eigenvectors (that have nonzero eigenvalues) of the kernelized covariance matrix C without ever explicitly computing C nor any vector of length D. Suppose that we have a test point $z \in \mathbb{R}^d$ and we want to efficiently compute the principal coordinate of $\Phi(z)$ projected onto a principal component $v = \sum_{j=1}^n a_j \Phi(X_j) = \Phi(X)^\top a$. Assume that we have already computed a. Describe how to compute $\Phi(z)$'s principal coordinate quickly, without ever computing a vector of length $\Theta(D)$ or doing a computation taking $\Theta(D)$ time. (Recall our assumption that kernel function computations are much, much faster than that.)

(a) As v is an eigenvector of C with a nonzero eigenvalue, there exists a nonzero scalar λ such that $Cv = \lambda v$. Substituting the formula for C into this expression gives us

$$\begin{aligned} \lambda v &= \frac{1}{n}\Phi(X)^\top\Phi(X)v \\ v &= \Phi(X)^\top \frac{1}{\lambda n}\Phi(X)v. \end{aligned}$$

So if we set $a = \frac{1}{\lambda n}\Phi(X)v$, the claim follows.

(b) Substituting $v = \Phi(X)^\top a$ and the formula for C into $Cv = \lambda v$ gives

$$\frac{1}{n}\Phi(X)^\top\Phi(X)\Phi(X)^\top a = \lambda\Phi(X)^\top a$$

Pre-multiplying both sides by $\Phi(X)$ gives

$$\begin{aligned} \frac{1}{n}\Phi(X)\Phi(X)^\top\Phi(X)\Phi(X)^\top a &= \lambda\Phi(X)\Phi(X)^\top a \\ K^2 a &= \lambda n K a. \end{aligned}$$

(c) As $\lambda_a a = Ka = \Phi(X)\Phi(X)^\top a$, we have $Cv = \frac{1}{n}\Phi(X)^\top\Phi(X)\Phi(X)^\top a = \frac{1}{n}\Phi(X)^\top Ka = \frac{\lambda_a}{n}\Phi(X)^\top a = \frac{\lambda_a}{n}v$. So v is an eigenvector of C with eigenvalue $\frac{\lambda_a}{n}$.

(d) The principal coordinate of $\Phi(z)$ on principal component v is $\Phi(z)^\top v$, so

$$\Phi(z)^\top \sum_{j=1}^{n} a_j \Phi(X_j) = \sum_{j=1}^{n} a_j k(z, X_j).$$

Q2. [16 pts] Performing PCA by Hand

Let's do principal components analysis (PCA)! Consider this sample of six points $X_i \in \mathbb{R}^2$.

$$\left\{\begin{bmatrix}0\\0\end{bmatrix}, \begin{bmatrix}0\\1\end{bmatrix}, \begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}2\\1\end{bmatrix}, \begin{bmatrix}2\\2\end{bmatrix}\right\}.$$

(a) [3 pts] Compute the mean of the sample points and write the centered design matrix $\dot{X}$.

The sample mean is

$$\mu = \frac{1}{6}\sum_{i=1}^{6} X_i = \begin{bmatrix}1\\1\end{bmatrix}.$$

By subtracting the mean from each sample, we form the centered design matrix

$$\dot{X} = \begin{bmatrix}-1 & -1\\ -1 & 0\\ 0 & -1\\ 0 & 1\\ 1 & 0\\ 1 & 1\end{bmatrix}.$$

(b) [6 pts] Find all the principal components of this sample. Write them as **unit** vectors.

The principal components of our dataset are the eigenvectors of the matrix

$$\dot{X}^\top \dot{X} = \begin{bmatrix}4 & 2\\ 2 & 4\end{bmatrix}.$$

The characteristic polynomial of this symmetric matrix is

$$\det(sI - X^\top X) = \det\begin{bmatrix}s-4 & -2\\ -2 & s-4\end{bmatrix} = (s-4)(s-4) - (-2)(-2)$$
$$= s^2 - 8s + 12 = (s-6)(s-2).$$

Hence the eigenvalues of $\dot{X}^\top \dot{X}$ are $\lambda_1 = 2$ and $\lambda_2 = 6$. With these eigenvalues, we can compute the eigenvectors of this matrix as follows. (Or you could just guess them and verify them.)

$$\begin{bmatrix}\lambda_1 - 4 & -2\\ -2 & \lambda_1 - 4\end{bmatrix} v_1 = \begin{bmatrix}-2 & -2\\ -2 & -2\end{bmatrix} v_1 = \begin{bmatrix}0\\0\end{bmatrix} \Rightarrow v_1 = \begin{bmatrix}1/\sqrt{2}\\ -1/\sqrt{2}\end{bmatrix}$$

$$\begin{bmatrix}\lambda_2 - 4 & -2\\ -2 & \lambda_2 - 4\end{bmatrix} v_2 = \begin{bmatrix}2 & -2\\ -2 & 2\end{bmatrix} v_2 = \begin{bmatrix}0\\0\end{bmatrix} \Rightarrow v_2 = \begin{bmatrix}1/\sqrt{2}\\ 1/\sqrt{2}\end{bmatrix}$$

(c) [4 pts]

- Which of those two principal components would be preferred if you use only one? [1 pt]
- What information does the PCA algorithm use to decide that one principal components is better than another? [1 pt]
- From an optimization point of view, why do we prefer that one? [2 pts]

We choose $v_2 = [1/\sqrt{2} \quad 1/\sqrt{2}]^\top$ first.

PCA picks the principal component with the largest eigenvalue.

We prefer it because it maximizes the variance of the sample points after they are projected onto a line parallel to v_2. (Alternatively, because it minimizes the sum of squares of distances from each sample point to its projection on that line.)

(d) [3 pts] Compute the **vector** projection of each of the **original** sample points (**not** the centered sample points) onto your **preferred principal component**. By "vector projection" we mean that the projected points are still in $\mathbb{R}^2$. (Don't just give us the principal coordinate; give us the projected point.)

The vector projection of X_i onto a direction w is

$$\tilde{X}_i = \frac{X_i^\top w}{\|w\|_2^2} w.$$

If w is a unit vector, we can just write $\tilde{X}_i = (X_i^\top w)w$.

For our sample and our chosen principal component $w = v_2$, the projections are

$$\tilde{X}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \tilde{X}_2 = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}, \tilde{X}_3 = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}, \tilde{X}_4 = \begin{bmatrix} 3/2 \\ 3/2 \end{bmatrix}, \tilde{X}_5 = \begin{bmatrix} 3/2 \\ 3/2 \end{bmatrix}, \tilde{X}_6 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

Q4. [15 pts] Backpropagation on an Arithmetic Expression

Consider an arithmetic network with the inputs a, b, and c, which computes the following sequence of operations, where $s(\gamma) = \dfrac{1}{1+e^{-\gamma}}$ is the logistic (sigmoid) function and $r(\gamma) = \max\{0, \gamma\}$ is the hinge function used by ReLUs.

$$d = ab \quad e = s(d) \quad f = r(a) \quad g = 3a \quad h = 2e + f + g \quad i = ch \quad j = f + i^2$$

We want to find the partial derivatives of j with respect to every other variable a through i, in backpropagation style. This means that for each variable z, we want you to write $\partial j/\partial z$ in two forms: (1) in terms of derivatives involving each variable that *directly* uses the value of z, and (2) in terms of the inputs and intermediate values $a \ldots i$, as simply as possible but with no derivative symbols. For example, we write

$$\frac{\partial j}{\partial i} = \frac{\mathrm{d}j}{\mathrm{d}i} = 2i \quad \text{(no chain rule needed for this one only)}$$
$$\frac{\partial j}{\partial h} = \frac{\partial j}{\partial i}\frac{\partial i}{\partial h} = 2ic \quad \text{(chain rule, then backprop the derivative expressions)}$$

(a) [15 pts] Now, please write expressions for $\partial j/\partial g$, $\partial j/\partial f$, $\partial j/\partial e$, $\partial j/\partial d$, $\partial j/\partial c$, $\partial j/\partial b$, and $\partial j/\partial a$ as we have written $\partial j/\partial h$ above. If they are needed, express the derivative $s'(\gamma)$ in terms of $s(\gamma)$ and express the derivative $r'(\gamma)$ as the indicator function $\mathbf{1}(\gamma \geq 0)$. (*Hint: f is used in two places and a is used in three, so they will need a multivariate chain rule. It might help you to draw the network as a directed graph, but it's not required.*)

$$\frac{\partial j}{\partial g} = \frac{\partial j}{\partial h}\frac{\partial h}{\partial g} = 2ic$$
$$\frac{\partial j}{\partial f} = \frac{\partial j}{\partial h}\frac{\partial h}{\partial f} + \frac{\mathrm{d}j}{\mathrm{d}f} = 2ic + 1$$
$$\frac{\partial j}{\partial e} = \frac{\partial j}{\partial h}\frac{\partial h}{\partial e} = 4ic$$
$$\frac{\partial j}{\partial d} = \frac{\partial j}{\partial e}\frac{\partial e}{\partial d} = 4ic\, s(d)(1 - s(d))$$
$$\frac{\partial j}{\partial c} = \frac{\partial j}{\partial i}\frac{\partial i}{\partial c} = 2ih$$
$$\frac{\partial j}{\partial b} = \frac{\partial j}{\partial d}\frac{\partial d}{\partial b} = 4ica\, s(d)(1 - s(d))$$
$$\frac{\partial j}{\partial a} = \frac{\partial j}{\partial d}\frac{\partial d}{\partial a} + \frac{\partial j}{\partial f}\frac{\partial f}{\partial a} + \frac{\partial j}{\partial g}\frac{\partial g}{\partial a} = 4icb\, s(d)(1 - s(d)) + (2ic + 1) \cdot \mathbf{1}(a \geq 0) + 6ic$$

Q6. [20 pts] Quadratic Regression

You are given a design matrix $X \in \mathbb{R}^{n \times d}$, storing n training points with d features, and a vector $y \in \mathbb{R}^n$ of continuously-valued labels. As usual, let X_i be training point i, expressed as a column vector such that row i of X is $X_i^\top$. We want to perform least-squares regression with the quadratic regression function $h(z) = z^\top W z$ (no linear nor constant terms), where $W \in \mathbb{R}^{d \times d}$ is symmetric. The objective is to find the symmetric matrix W that minimizes the least-squares cost function

$$J(W) = \sum_{i=1}^{n} \left(X_i^\top W X_i - y_i \right)^2 .$$

(*Note: If you get stuck on one part, try the other parts; each part is independent of solving the others.*)

(a) [6 pts] Find $\nabla_W J(W)$. (*Hint: It should be a $d \times d$ matrix like W. If you don't remember how to take a derivative with respect to a matrix, try writing $X_i^\top W X_i$ as a summation, but your final answer should have just one summation, over i.*)

$$\begin{aligned}
\nabla_W J(W) &= \sum_{i=1}^{n} \nabla_W (X_i^\top W X_i - y_i)^2 \\
&= \sum_{i=1}^{n} 2(X_i^\top W X_i - y_i) \cdot \nabla_W (X_i^\top W X_i - y_i) \\
&= 2 \sum_{i=1}^{n} (X_i^\top W X_i - y_i) X_i X_i^\top .
\end{aligned}$$

(b) [4 pts] Suppose that X has rank n. (That is, the training points are linearly independent and $n \leq d$.) Let I_i be the **row vector** that is row i of the $n \times n$ identity matrix; hence, its entries are zero except a 1 at position i. Observe that we can write training point i as the row vector $X_i^\top = I_i X$. Let X^+ be the Moore–Penrose pseudoinverse of X. As a prelude to part (c), show that $X_i^\top X^+ = I_i$. (*Hint: What do you know about pseudoinverses?*)

By postmultiplying both sides of $X_i^\top = I_i X$ by X^+, we have $X_i^\top X^+ = I_i X X^+ = I_i$. (As X is an $n \times d$ matrix with rank n, $XX^+ = I$.)

(c) [5 pts] Again suppose that X has rank n. Write the labels as a diagonal $n \times n$ matrix

$$Y = \operatorname{diag}(y_i) = \begin{bmatrix} y_1 & 0 & \dots & 0 \\ 0 & y_2 & & 0 \\ \vdots & & \ddots & \\ 0 & \dots & & y_n \end{bmatrix}.$$

Using (b), show that the cost function $J(W)$ is minimized by $W^* = X^+ Y X^{+\top}$, where $X^{+\top}$ denotes the transpose of X^+.

For each $i \in [1, n]$,

$$X_i^\top W^* X_i = X_i^\top X^+ Y X^{+\top} X_i = I_i Y I_i^\top = y_i.$$

Therefore, $J(W^*) = \sum_{i=1}^{n} (X_i^\top W^* X_i - y_i)^2 = 0$. As $J(W)$ is nonnegative for all W, no matrix has a lower cost than W^*, so W^* is optimal.

Note: we will also accept answers that show that $\nabla_W J(W)$ is zero at $W = W^*$ (instead of showing that $J(W^*) = 0$).

(d) [5 pts] Suppose the ground truth for the labels is $g(z) = z^\top A z$, where $A \in \mathbb{R}^{d \times d}$ is a fixed, symmetric matrix. Each training label is drawn from this ground truth with random Gaussian noise; that is, $y_i = g(X_i) + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, 1)$. Assume that the regression function we learn is $h(z) = z^\top W^* z$ where W^* is given in part (c). Show that the **bias** of this regression method at a test point z is

$$\text{bias}(z) = |z^\top (X^+ \operatorname{diag}(X_i^\top A X_i) X^{+\top} - A) z|,$$

where $\text{diag}(X_i^\top A X_i)$ is a diagonal $n \times n$ matrix whose diagonal values are $X_i^\top A X_i$.

$$\text{bias}(z) = |E[h(z)] - g(z)| = |E[z^\top X^+ Y X^{+\top} z] - z^\top A z| = |z^\top (X^+ E[Y] X^{+\top} - A) z|.$$

$E[Y]$ is a diagonal matrix whose diagonal values are $E[y_i] = E[g(X_i) + \epsilon_i] = g(X_i) = X_i^\top A X_i$, so

$$\text{bias}(z) = |z^\top (X^+ \text{diag}(X_i^\top A X_i) X^{+\top} - A) z|.$$